

Генерация текста

План

- Задачи генерации
- Уменьшение размера словаря, токенизация
- Text CNN
- RNN, LSTM

Зачем это нужно?

Автодополнение слова

В лесу родилась елочка

Автодополнение фразы

Почему птицы летают
поют
летят клином

Диалоговые системы

– Какая погода в Москве?

В Москве сейчас 8 градусов –

Постановка задачи

Текст должен удовлетворять требованиям:

- Логическая связность
- Соблюдение языковых норм

Можно попытаться задать правила вручную.

Но правил слишком много, поэтому ничего хорошего не выйдет.

Постановка задачи

Будем учиться имитировать человеческую речь.

Для этого научимся оценивать вероятность текстов.

$p(\text{В лесу родилась елочка}) \quad \vee \quad p(\text{У лукоморья дуб зеленый})$

$$\frac{\parallel 1}{|\text{dataset}|}$$

$$\frac{1 \parallel}{|\text{dataset}|}$$

Работать с текстом как с одним целым невозможно!

Постановка задачи

Разделим текст на слова.

x – текст из m слов.

Обучим модель оценивать вероятность набора слов.

$$p(x) = p(x_1, \dots, x_m)$$

Заменим совместную вероятность на произведение условных вероятностей.

$$p(x_1, \dots, x_m) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots \cdot p(x_m | x_1, \dots, x_{m-1}) = \prod_{i=1}^m p(x_i | x_{<i})$$

Постановка задачи

$$p(x_1, \dots, x_m) = \prod_{i=1}^m p(x_i | x_{<i})$$

Достаточно обучить модель оценивать вероятность $p(x_i | x_{<i})$.

Все еще сложно, потому что надо учитывать все предыдущие слова.

Упростим задачу – будем смотреть только на предыдущие n слов.

$$p(x_1, \dots, x_m) \approx \prod_{i=1}^m p(x_i | x_{i-1}, \dots, x_{i-n})$$

Такая модель называется n -граммной.

Уменьшение размера словаря

- Удаление стоп-слов
- Лемматизация
- Стемминг
- Изменение способа токенизации

Токенизация

- Любой язык содержит много уникальных слов.
(Около **200к** в русском и **170к** в английском)
- Токенизация по **словам** приводит к получению **огромного словаря**.

"В", "лесу", "родилась", "елочка".

Токенизация

- Букв в алфавите гораздо меньше.
(33 в русском языке и 26 в английском)
- Токенизируя по **буквам** можно значительно **уменьшить словарь**.

"В", " ", "л", "е", "с", "у", " ", "р", "о", "д", "и", "л", "а", "с", "ь".

Задачи алгоритма токенизации

- Возможность токенизировать любой текст.
- Размер словаря не превышает определенное значение.
- Токены несут в себе как можно больше смысла.
- Текст токенизируется наименьшим числом токенов.

Алгоритмы токенизации

- Byte-Pair Encoding
- WordPiece
- Unigram

Byte-Pair Encoding (BPE)

Корпус: ["вол", "стол", "стул", "ствол"]

- Стартовый словарь – все возможные буквы.
Словарь: ["в", "о", "л", "с", "т", "у"]
- Итеративно увеличиваем словарь, склеивая пары токенов в один.
- Склеиваем ту пару, которая встречается чаще всего в корпусе.

Byte-Pair Encoding: алгоритм

Корпус: ["вол", "стол", "стул", "ствол"]

Шаг 1

Словарь: ["в", "о", "л", "с", "т", "у"]

Пары	Встречаемость
с, т	3
о, л	3
в, о	2

Byte-Pair Encoding: алгоритм

Шаг 1

Пары	Встречаемость
с, т	3
о, л	3
в, о	2

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст"]

Добавляем "ст"



Byte-Pair Encoding: алгоритм

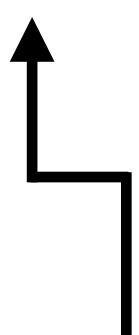
Шаг 2

Пары	Встречаемость
о, л	3
в, о	2
ст, о	1
ст, у	1
ст, в	1

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"ол"]

Добавляем "ол"



Byte-Pair Encoding: алгоритм

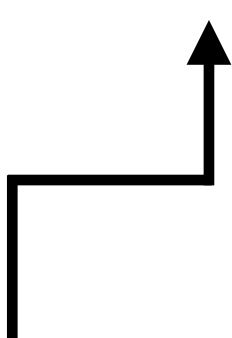
Шаг 3

Пары	Встречаемость
В, О	2
В, ОЛ	2
СТ, ОЛ	1

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"ол", "вол"]

Добавляем "вол"



Byte-Pair Encoding: алгоритм

Шаг 4

Пары	Встречаемость
В, О	2
СТ, ВОЛ	1

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"ол", "вол", "ВО"]

Добавляем "во"



Byte-Pair Encoding: алгоритм

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"ол", "вол", "во"]

Останавливаемся, когда словарь достиг нужного размера.

WordPiece токенизация

Корпус: ["вол", "стол", "стул", "ствол"]

- Стартовый словарь – все возможные буквы.

Словарь: ["в", "о", "л", "с", "т", "у"]

- Итеративно увеличиваем словарь, склеивая пары токенов в один.

- Для каждой пары токенов (t_i, t_j) считаем скор $\frac{p(\overline{t_i t_j})}{p(t_i)p(t_j)} = \frac{N(\overline{t_i t_j})}{N(t_i)N(t_j)}$

- Склеиваем пару с наибольшим скором.

Таким образом, мы выбираем ту пару, добавление которой больше всего увеличивает **правдоподобие** выборки

WordPiece: алгоритм

Шаг 1

Пары	score
с, т	$3 / (3 \cdot 3) = 1/3$
о, л	$3 / (3 \cdot 4) = 1/4$
в, о	$2 / (2 \cdot 3) = 1/3$

$$score(t_i, t_j) = \frac{N(\overline{t_i t_j})}{N(t_i)N(t_j)}$$

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст"]

Добавляем "ст" 

WordPiece: алгоритм

Шаг 2

Пары	score
о, л	$3/(3 \cdot 4) = 1/4$
в, о	$2/(2 \cdot 3) = 1/3$
у, л	$1/(1 \cdot 3) = 1/3$

$$score(t_i, t_j) = \frac{N(\overline{t_i t_j})}{N(t_i)N(t_j)}$$

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"во"]

Добавляем "во"



WordPiece: алгоритм

Шаг 3

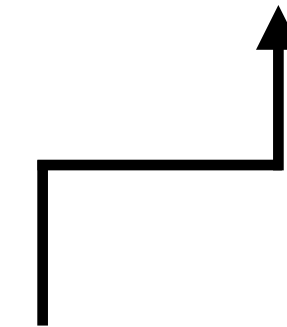
Пары	score
во, л	$2/(2 \cdot 4) = 1/4$
о, л	$3/(3 \cdot 4) = 1/4$
у, л	$1/(1 \cdot 3) = 1/3$

$$score(t_i, t_j) = \frac{N(\overline{t_i t_j})}{N(t_i)N(t_j)}$$

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"во", "ул"]

Добавляем "ул"



WordPiece: алгоритм

Шаг 4

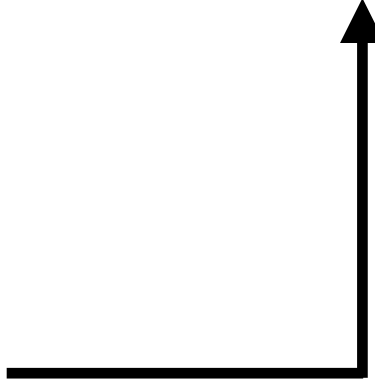
Пары	score
т, ул	$1/(3 \cdot 1) = 1/3$
во, л	$2/(2 \cdot 4) = 1/4$
о, л	$3/(3 \cdot 4) = 1/4$

$$score(t_i, t_j) = \frac{N(\overline{t_i t_j})}{N(t_i)N(t_j)}$$

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"во", "ул", "тул"]

Добавляем "тул"



WordPiece: алгоритм

Корпус: ["вол", "стол", "стул", "ствол"]

Словарь: ["в", "о", "л", "с", "т", "у", "ст",
"во", "ул", "тул"]

Останавливаемся, когда словарь достиг нужного размера.

Unigram

- Unigram работает в противоположную сторону относительно BPE и WordPiece
- Исходный словарь состоит из всех возможных токенов
- На каждом шаге удаляем один токен

Unigram: идея

- Unigram (1-грамм) модель считает, что все токены независимы

$$p(x) = p(x_1, \dots, x_m) = \prod_{i=1}^m p(x_i) \quad p(x_i) = \frac{N(x_i)}{\sum_{t \in V} N(t)}$$

- Мы хотим максимизировать правдоподобие корпуса текстов
- Будем **удалять** из словаря те **токены**, которые **минимально уменьшают правдоподобие** Unigram модели

Unigram: идея

- Unigram (1-грамм) модель считает, что все токены независимы

$$p(x) = p(x_1, \dots, x_m) = \prod_{i=1}^m p(x_i) \quad p(x_i) = \frac{N(x_i)}{\sum_{t \in V} N(t)}$$

- Мы хотим максимизировать правдоподобие корпуса текстов
- Будем **удалять** из словаря те **токены**, которые **минимально уменьшают правдоподобие** Unigram модели
- Остановимся, когда размер словаря достигнет нужного размера

Unigram: алгоритм

Корпус: ["вол", "стол", "кол"]

Словарь: ["в", "о", "л", "с", "т", "к", "во", "ол", "ст", "то", "ко", "вол", "сто", "тол"]

Токен	в	о	л	с	т	к	во	ол	ст	то	ко	вол	сто	тол
Встречаемость	1	3	3	1	1	1	1	3	1	1	1	1	1	1

Всего: 20

Unigram: алгоритм

Корпус: ["вол", "стол", "кол"]

Словарь: ["в", "о", "л", "с", "т", "к", "во", "ол", "ст", "то", "ко", "вол", "сто", "тол"]

Токен	в	о	л	с	т	к	во	ол	ст	то	ко	вол	сто	тол
Встречаемость	1	3	3	1	1	1	1	3	1	1	1	1	1	1

Всего: 20

$$p(\text{"стол"}) \begin{cases} p(\text{"сто"}, \text{"л"}) = 1/20 \cdot 3/20 \\ p(\text{"с"}, \text{"тол"}) = 1/20 \cdot 1/20 \end{cases}$$

Удаление "с" и "тол" не уменьшит вероятность, значит их можно выбросить.

Unigram: алгоритм

Корпус: ["вол", "стол", "кол"]

Словарь: ["в", "о", "л", "т", "к", "во", "ол", "ст", "то", "ко", "вол", "сто"]

Токен	в	о	л	т	к	во	ол	ст	то	ко	вол	сто
Встречаемость	1	3	3	1	1	1	3	1	1	1	1	1

Всего: 18

Unigram: алгоритм

Корпус: ["вол", "стол", "кол"]

Словарь: ["в", "о", "л", "т", "к", "во", "ол", "ст", "то", "ко", "вол", "сто"]

Токен	в	о	л	т	к	во	ол	ст	то	ко	вол	сто
Встречаемость	1	3	3	1	1	1	3	1	1	1	1	1

Всего: 18

Что поменяется, если удалить "л"?

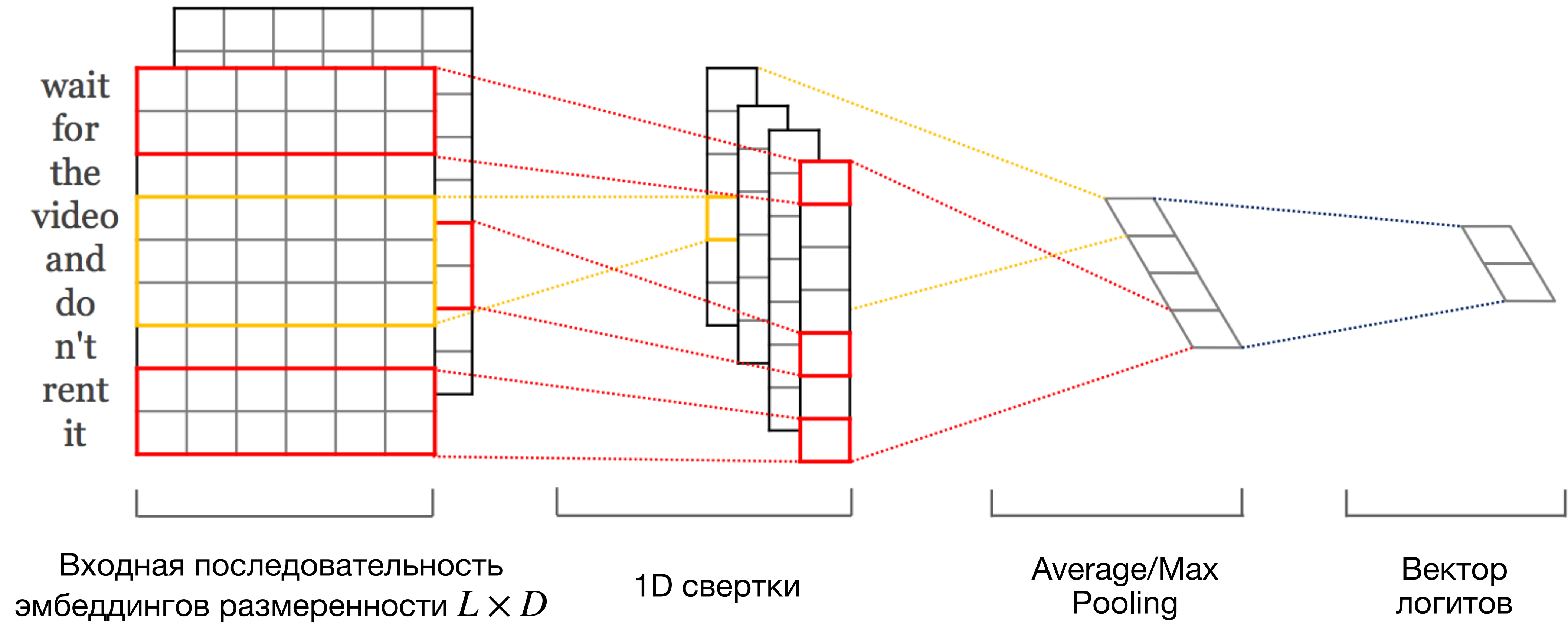
$$p(\text{"вол"}) \begin{cases} p(\text{"во"}, \text{"л"}) = 1/18 \cdot 3/18 \\ p(\text{"в"}, \text{"ол"}) = 1/18 \cdot 3/18 \end{cases}$$

$$p(\text{"кол"}) \begin{cases} p(\text{"ко"}, \text{"л"}) = 1/18 \cdot 3/18 \\ p(\text{"к"}, \text{"ол"}) = 1/18 \cdot 3/18 \end{cases}$$

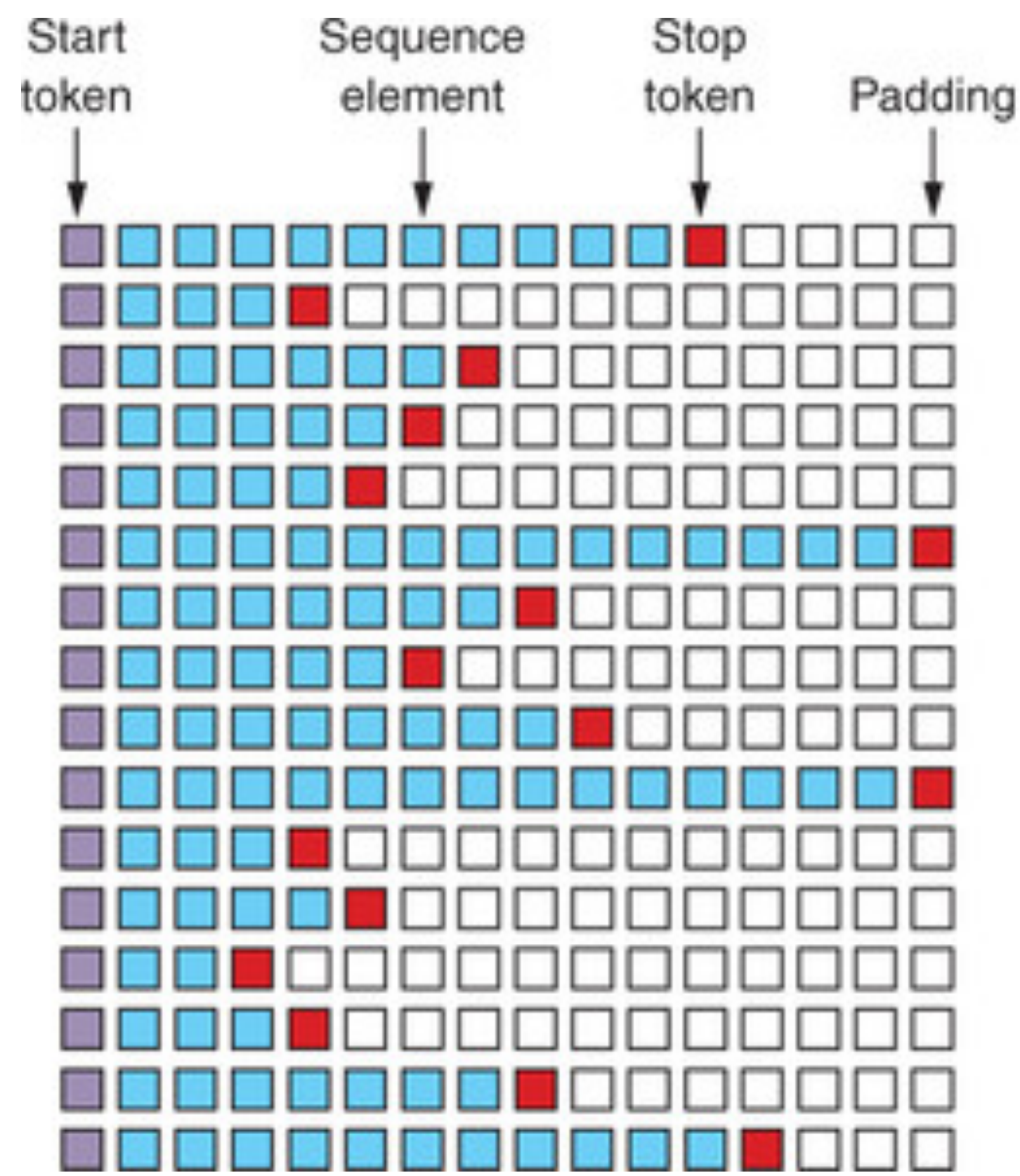
$$p(\text{"стол"}) \begin{cases} p(\text{"сто"}, \text{"л"}) = 1/18 \cdot 3/18 \\ p(\text{"ст"}, \text{"ол"}) = 1/18 \cdot 3/18 \end{cases}$$

Ничего не меняется => удаляем

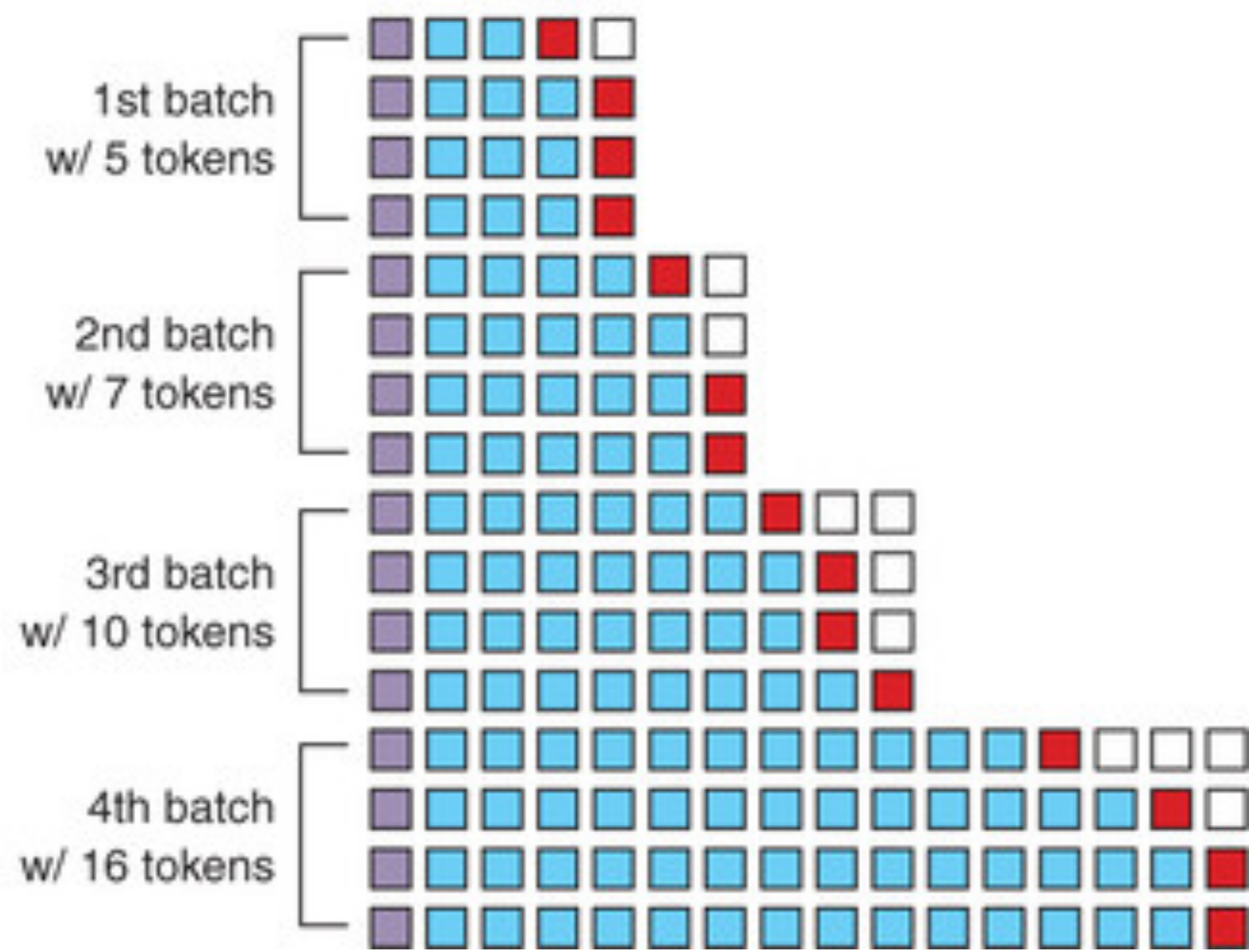
Text CNN



Токен паддинга

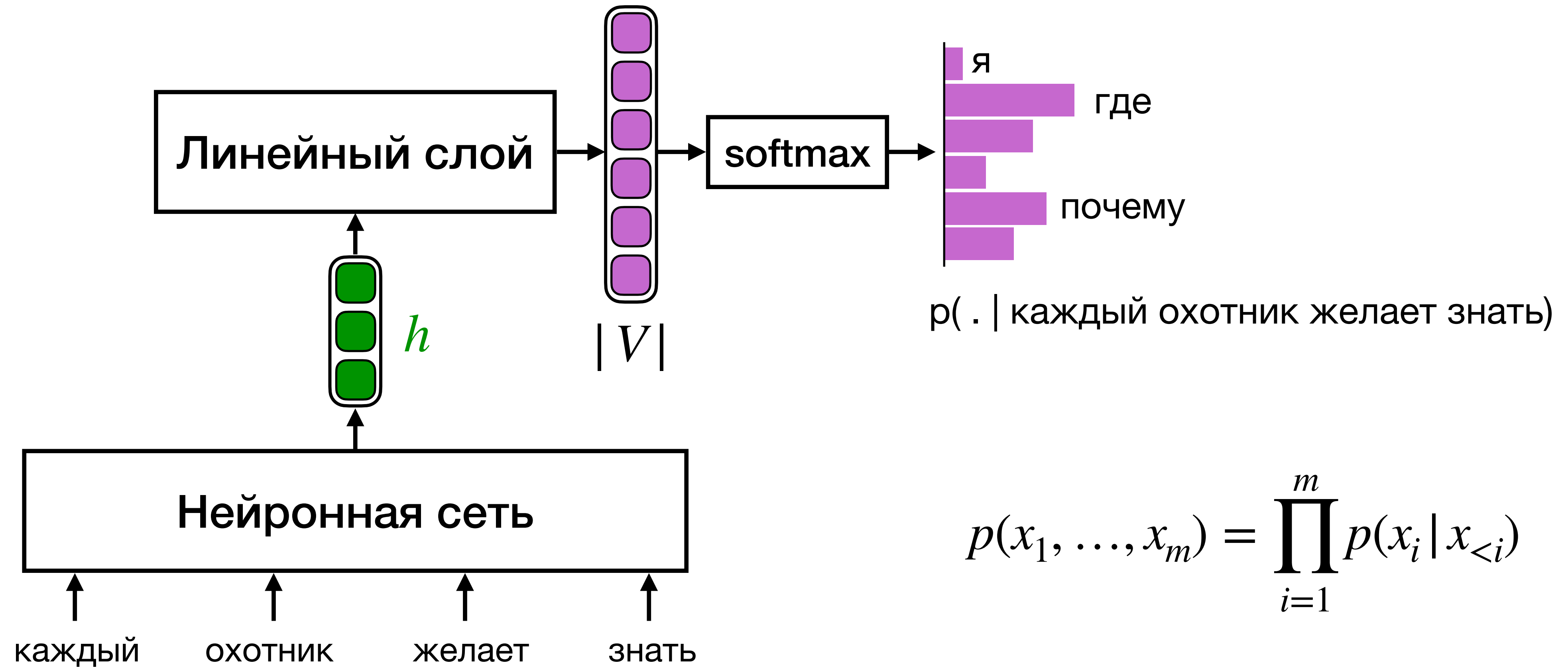


Датасет



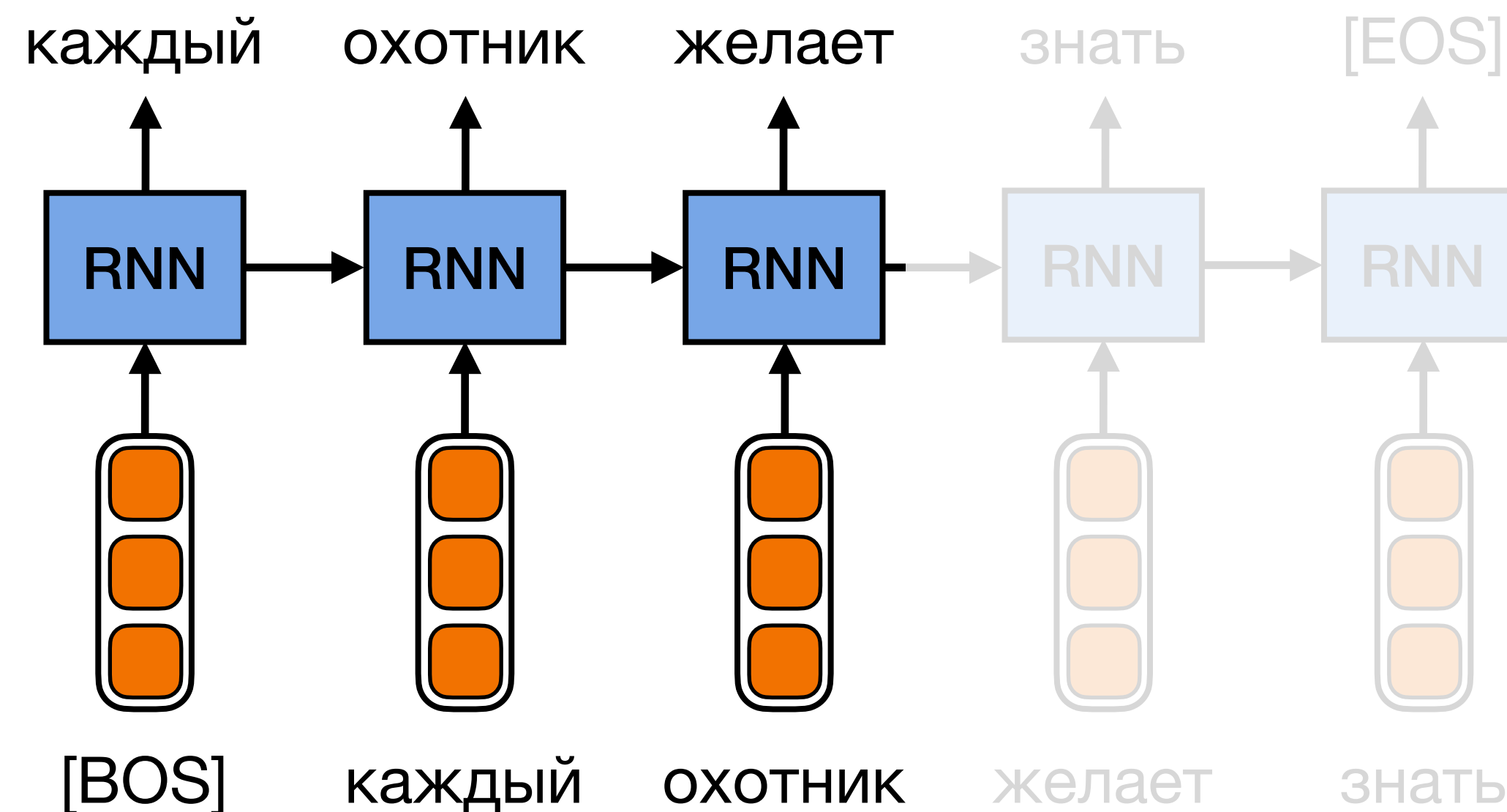
Разделение на батчи

Нейронные сети для генерации текста



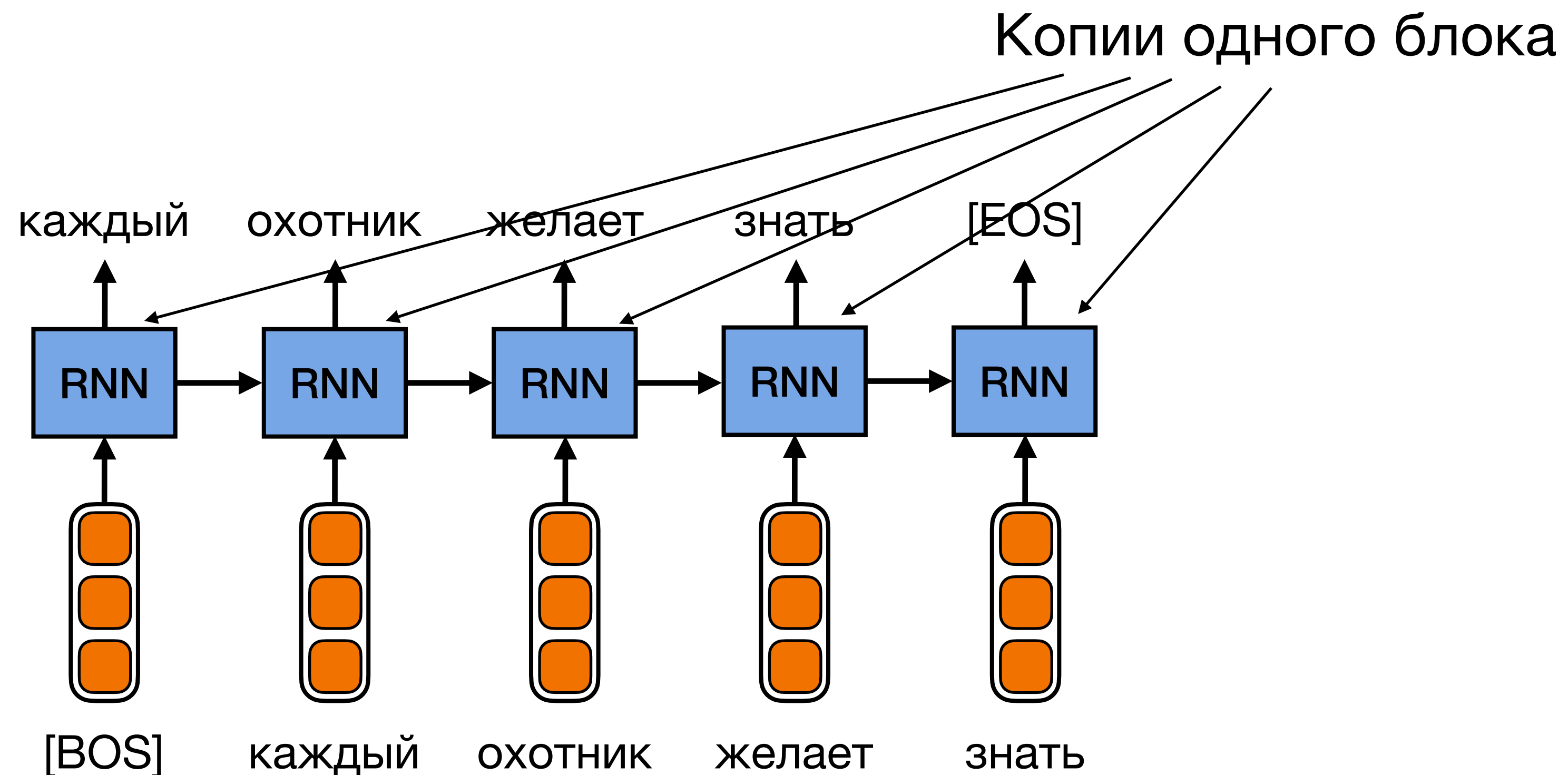
Recurrent Neural Networks (RNN)

- Разработаны для работы с последовательными данными.
- Каждый блок предсказывает следующий токен.
- Процесс генерации интуитивно понятен.

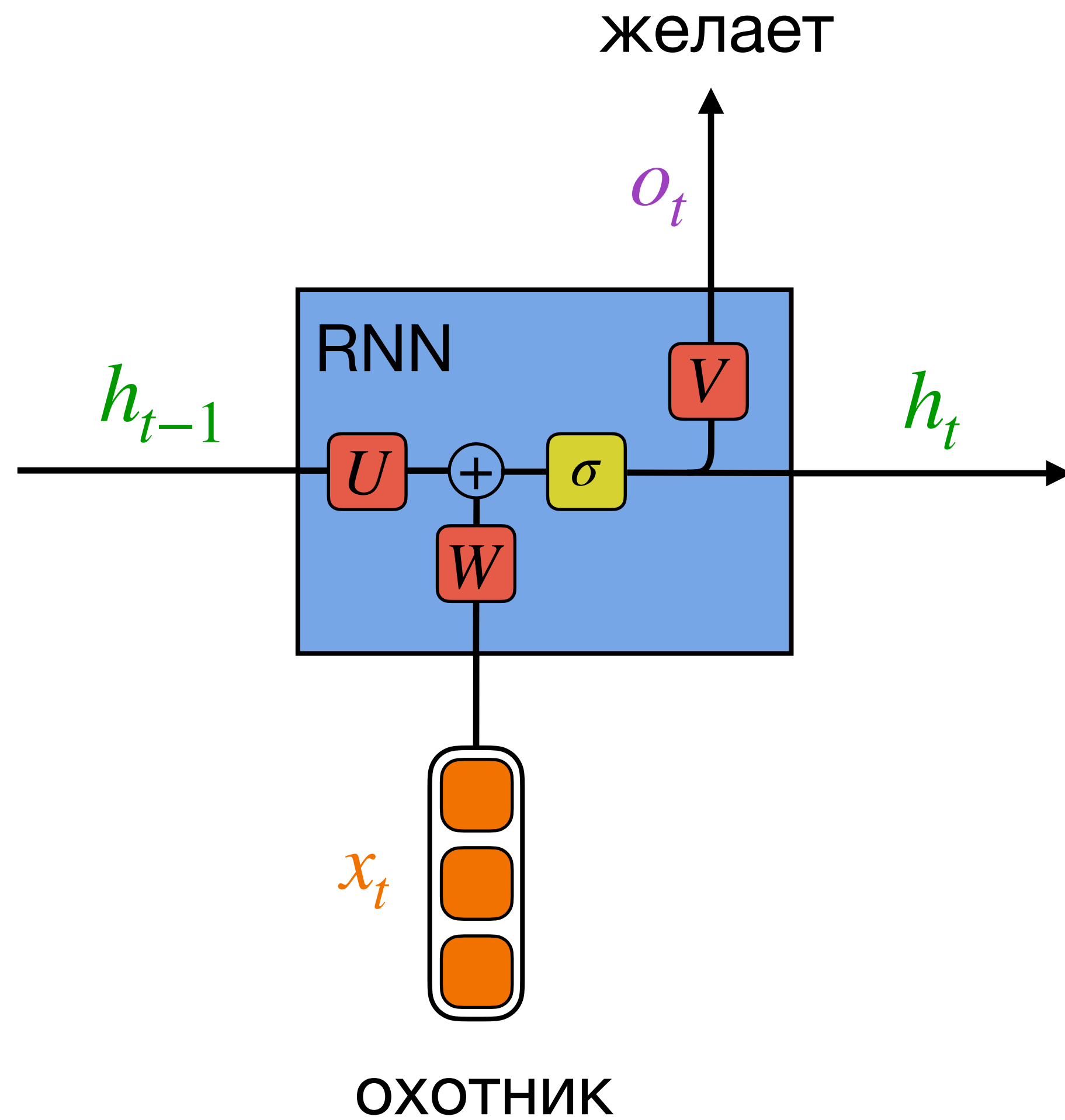


Recurrent Neural Networks (RNN)

- Разработаны для работы с последовательными данными.
- Каждый блок предсказывает следующий токен.



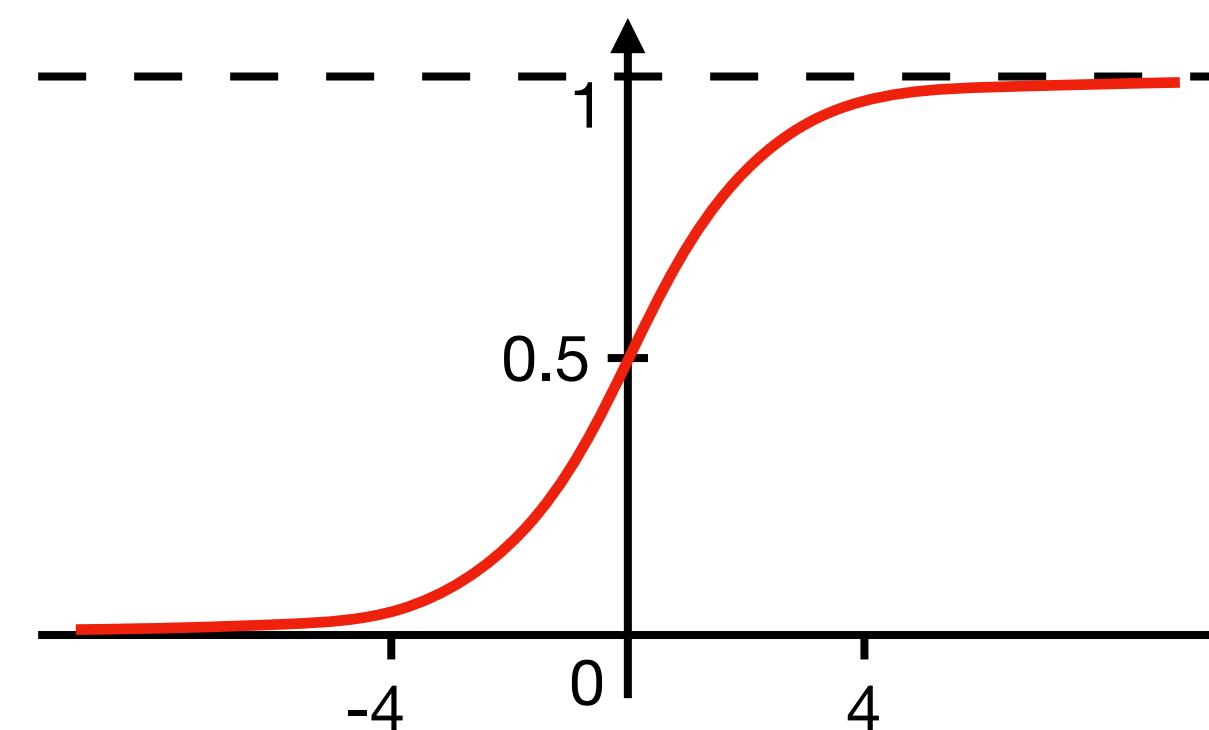
Блок RNN



$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



RNN: Обучение

$$p(x_1, \dots, x_m) = \prod_{t=1}^m p(x_t | x_{<t}) \rightarrow \max$$

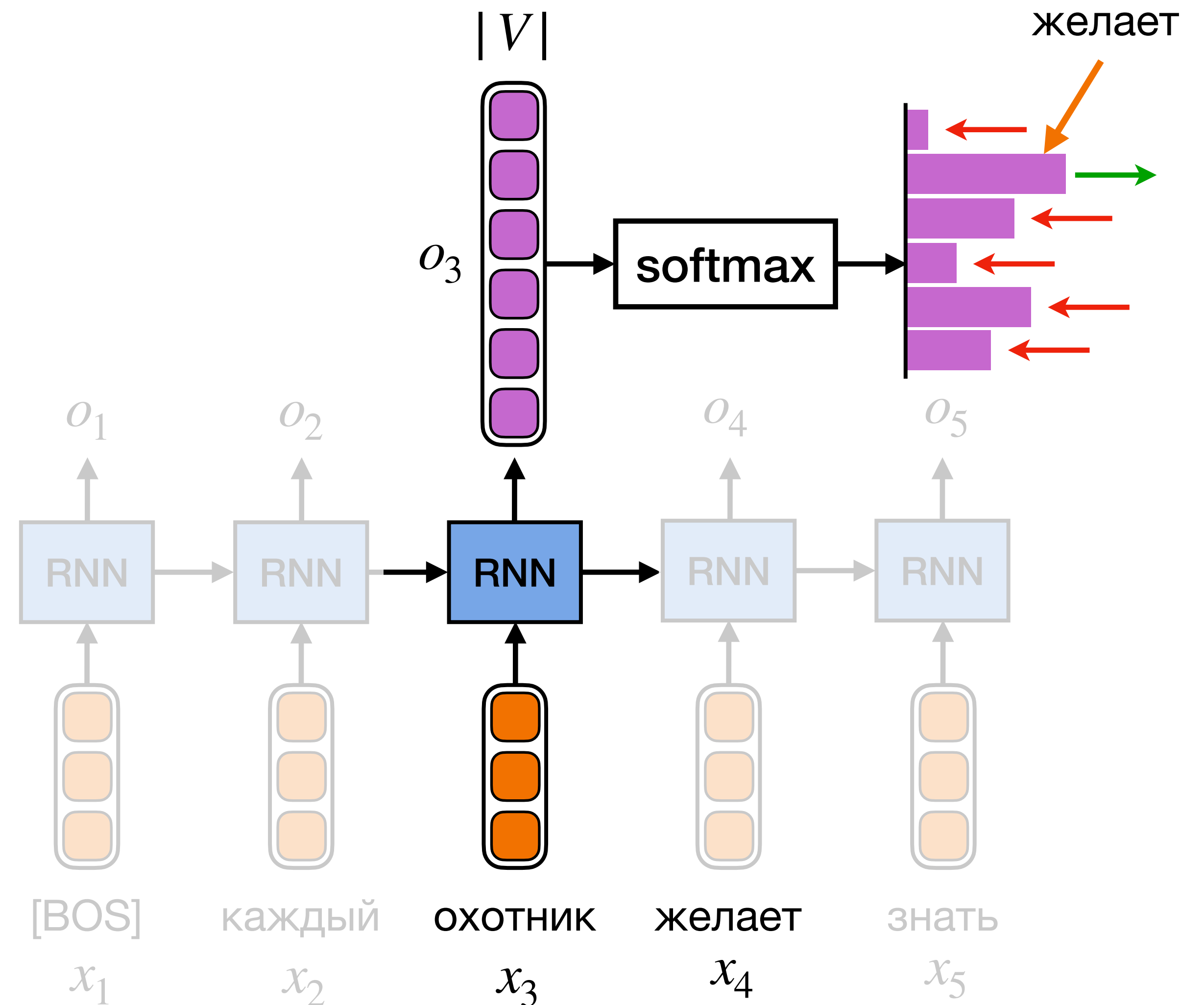
$$p(\cdot | x_{<t}) = \text{softmax}(o_t)$$

Накладывая логарифм и отрицание, получаем кросс-энтропию.

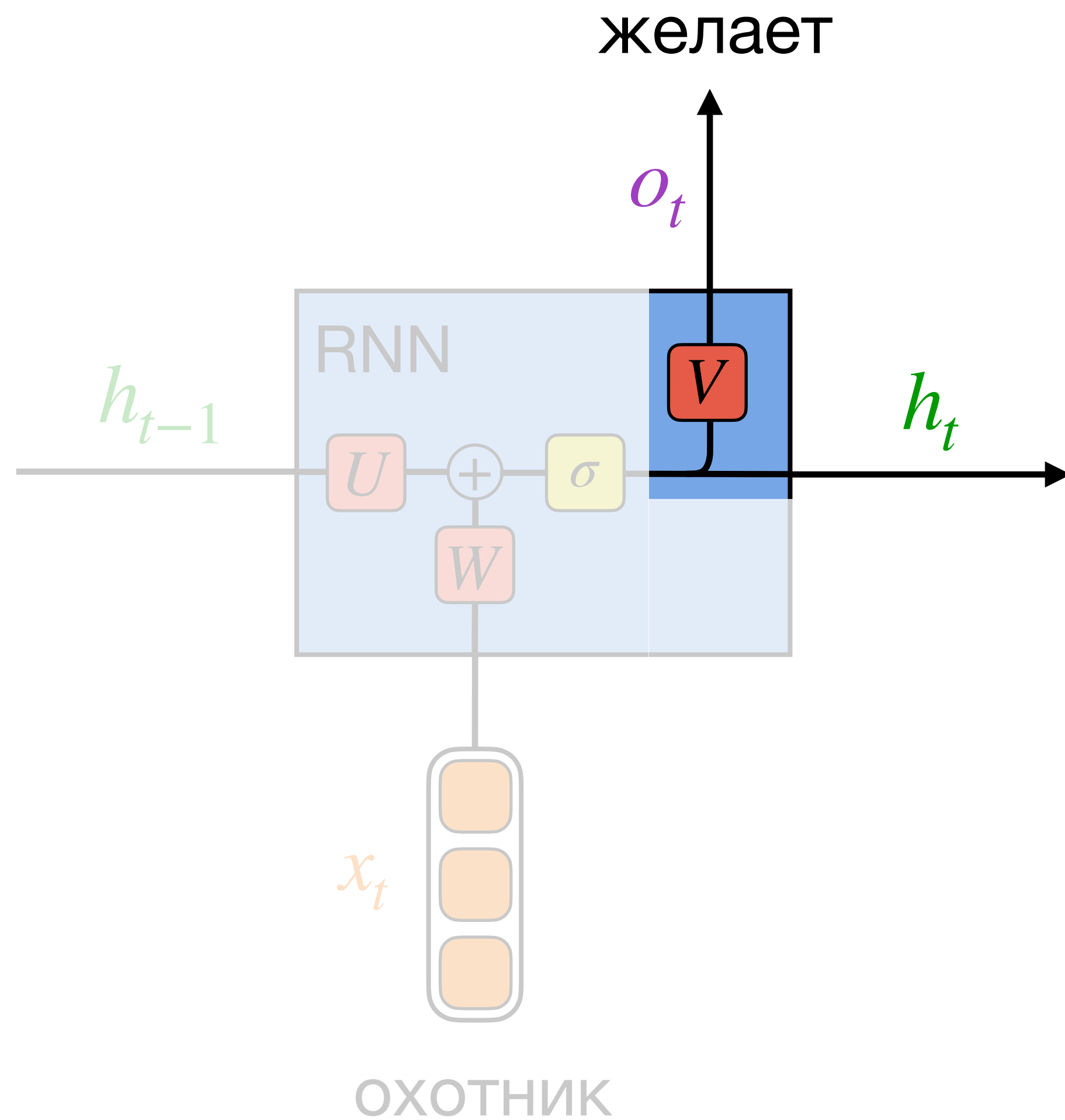
$$L(x) = - \sum_{t=1}^m \log p(x_t | x_{<t}) \rightarrow \min$$

Ошибка для всего корпуса.

$$L(X) = - \frac{1}{|X|} \sum_{x \in X} \sum_{t=1}^m \log p(x_t | x_{<t}) \rightarrow \min$$



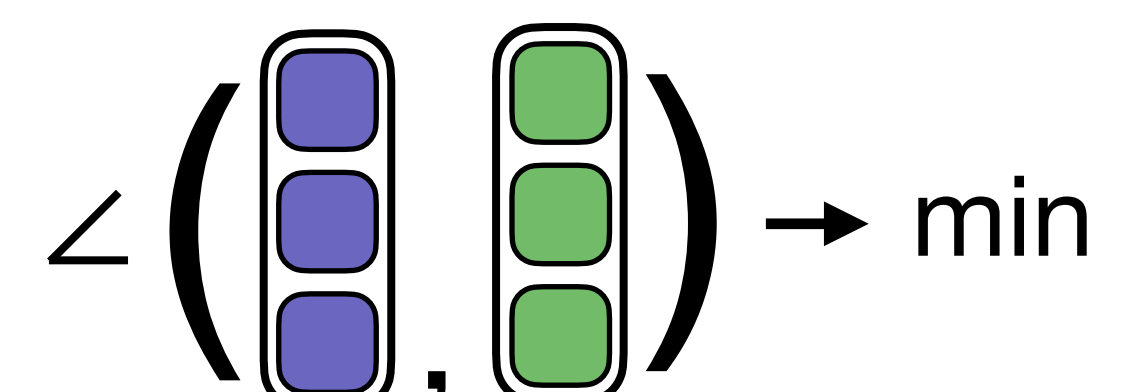
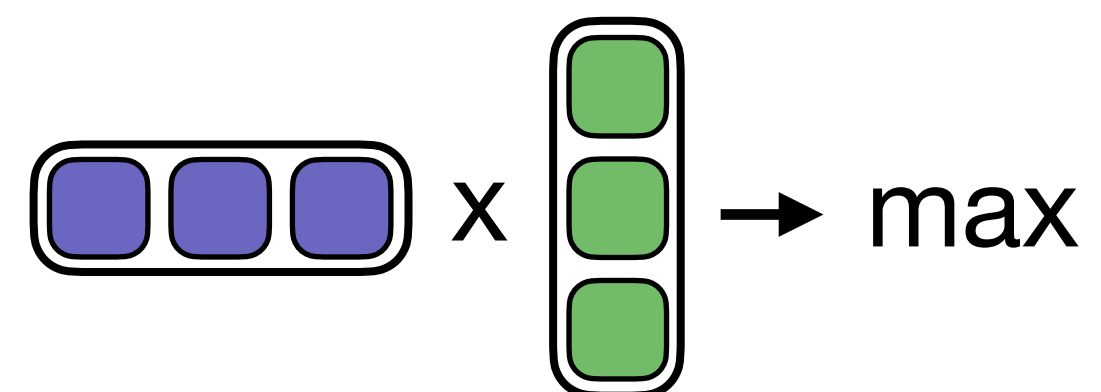
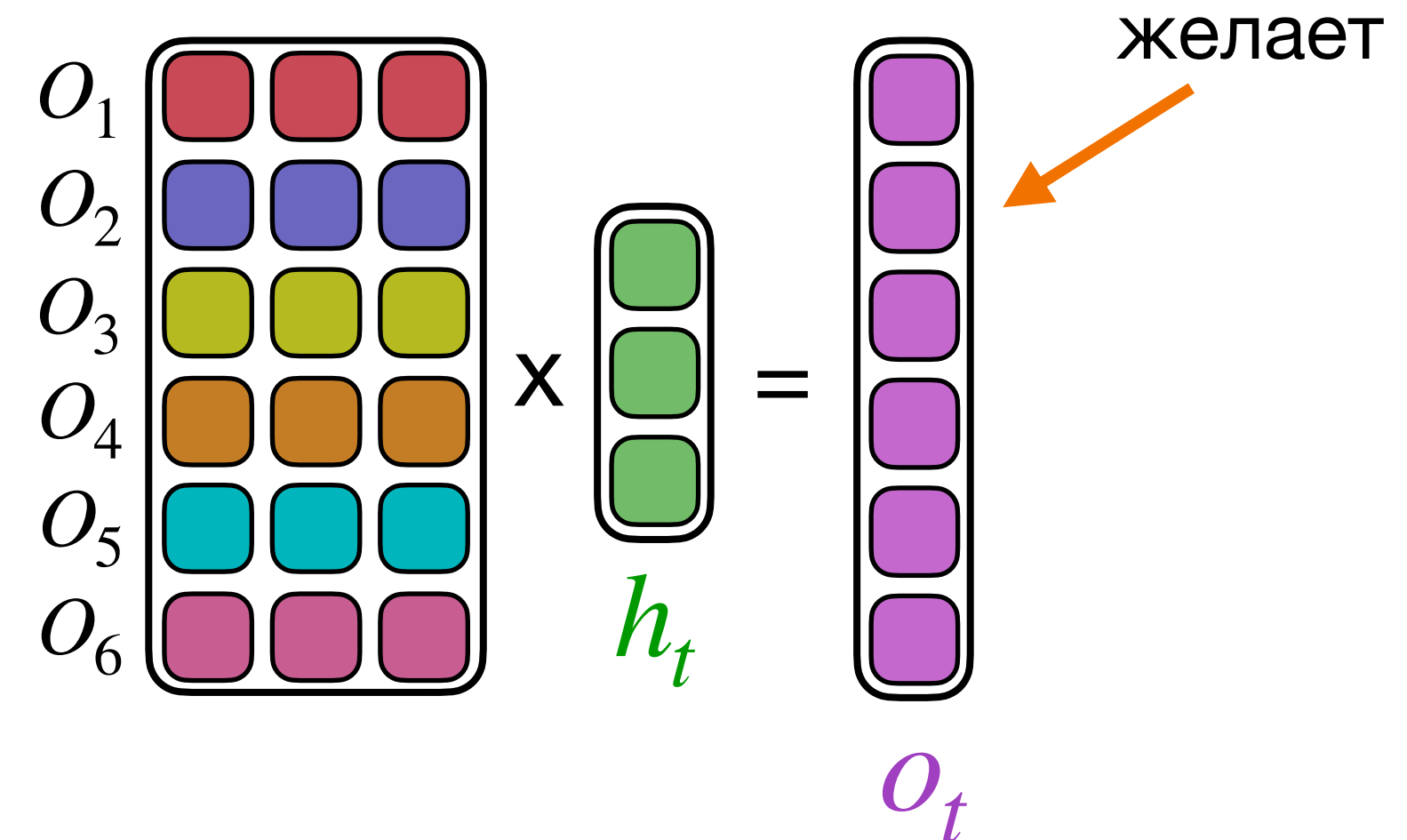
Как еще об этом можно думать?



$$o_t = Vh_t + b_v$$

Выходные эмбединги

ТОКЕНОВ



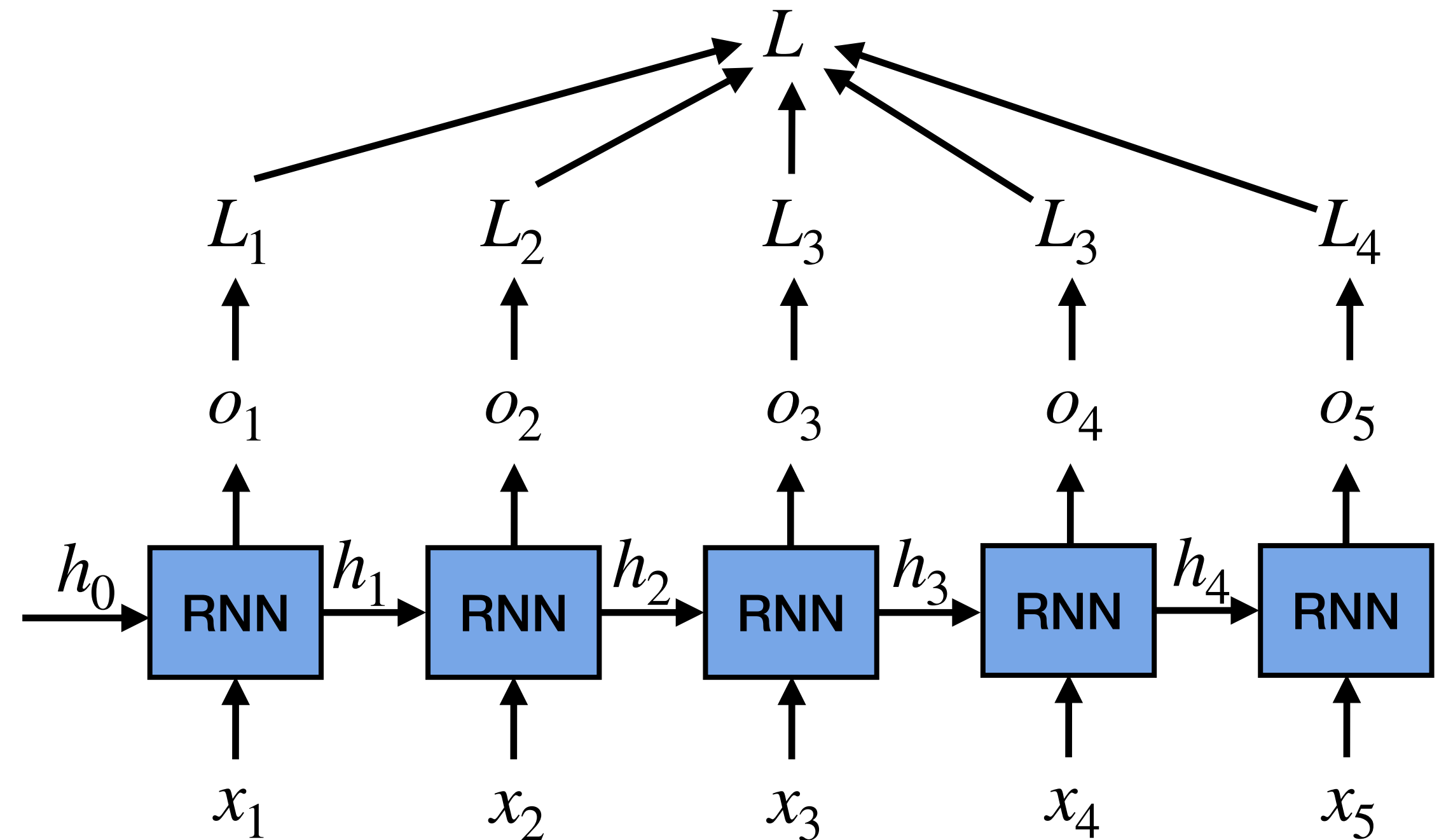
Градиенты RNN

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$



Градиенты RNN

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

chain rule

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

Градиенты RNN

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\frac{dh_t}{dU} = \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU}$$

Переходим от производных к частным производным

Градиенты RNN

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \boxed{\frac{dh_{t-1}}{dU}} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \boxed{\left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right)} \end{aligned}$$

Градиенты RNN

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right) = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \end{aligned}$$

Раскрываем скобки

Градиенты RNN

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right) = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} = \\ &= \sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \end{aligned}$$

Собираем все в одну сумму

Градиенты RNN

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Vh_t + b_v$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right) = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} = \\ &= \sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \end{aligned}$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \right]$$

Взрыв градиентов

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \right]$$

Серия умножений производных

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| > 1$$

- Происходит **взрыв градиента** $\frac{dL}{dU}$
- Модель расходится, NaN в весах

Решения:

- Регуляризация
- Уменьшение learning rate
- Gradient clipping

$$1. g \leftarrow \min \left(1, \frac{\text{max norm}}{\|g\|} \right) \cdot g$$

Правильный способ

$$2. g \leftarrow \text{clip}(g, -C, C)$$

Ленивый способ (меняет направление градиента)

Затухание градиентов

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \right]$$
$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| < 1$$

- Происходит **затухание градиента**
- Модель перестает учиться
- Модель не улавливает далекие зависимости!

Затухание градиентов – частая проблема RNN.

Ее нельзя починить трюками.

Затухание градиентов: почему возникает?

$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}}$$

Затухание градиентов: почему возникает?

$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

Поэлементное умножение

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}} = \left(\sigma(z_j) \odot (1 - \sigma(z_j)) \right) U$$

Затухание градиентов: почему возникает?

$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

Поэлементное умножение

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}} = \left(\sigma(z_j) \odot (1 - \sigma(z_j)) \right) U$$

Посмотрим на спектральную норму

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \underbrace{\| \sigma(z_j) \odot (1 - \sigma(z_j)) \|}_{< 1} \cdot \|U\|$$

т. к. $\sigma(z) \in [0, 1]$

Затухание градиентов: почему возникает?

$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

Поэлементное умножение

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}} = \left(\sigma(z_j) \odot (1 - \sigma(z_j)) \right) U$$

Посмотрим на спектральную норму

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \underbrace{\| \sigma(z_j) \odot (1 - \sigma(z_j)) \|}_{< 1} \cdot \|U\|$$

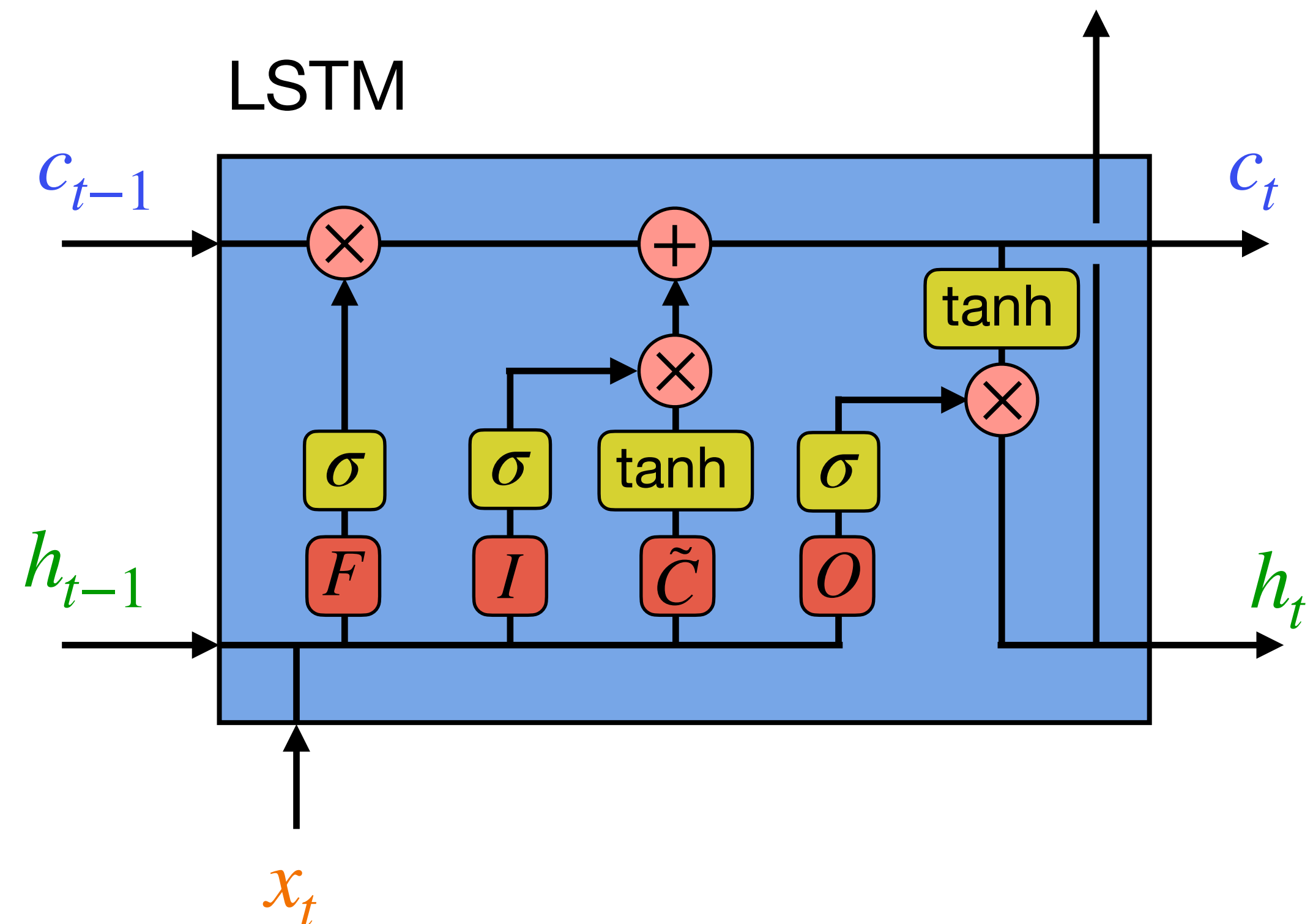
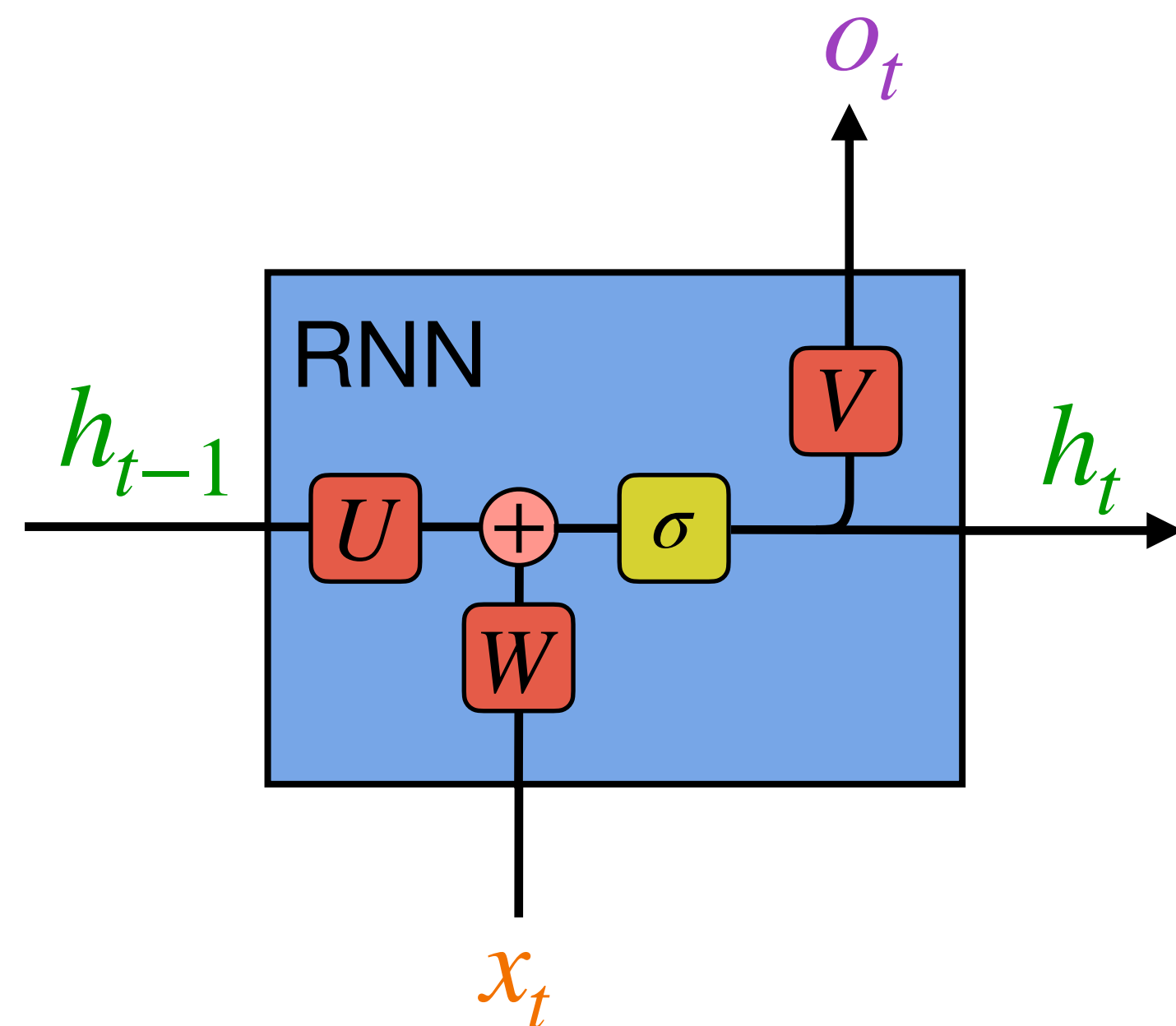
т. к. $\sigma(z) \in [0, 1]$

Если U – ортогональная ($UU^T = I$), то

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \| \sigma(z_j) \odot (1 - \sigma(z_j)) \| < 1$$

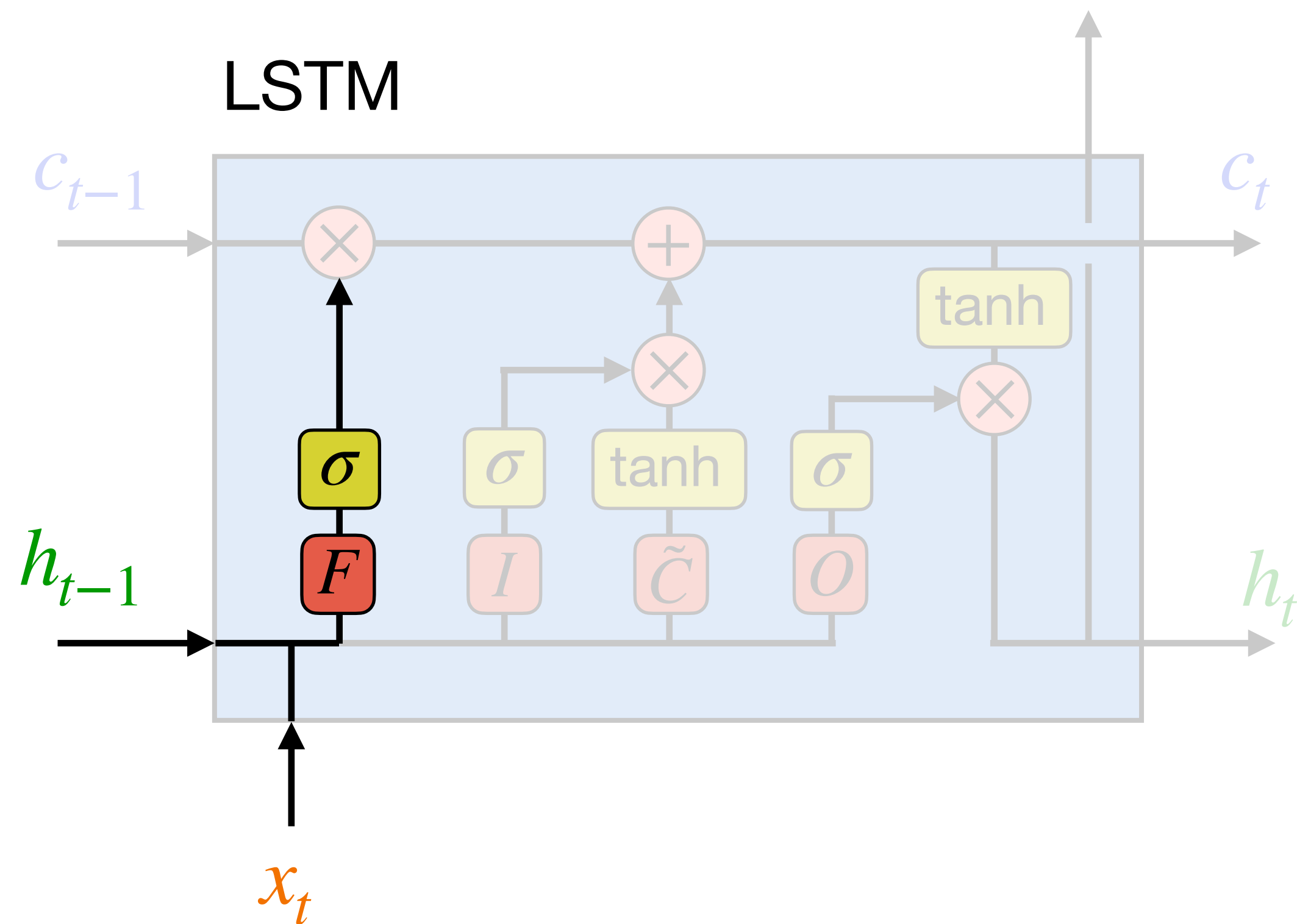
Long short-term memory (LSTM)

- В LSTM добавляется вектор памяти c_t
- Благодаря ему модель не забывает старую информацию
- Так же добавляется 3 фильтра



LSTM: фильтр забывания

Контролирует, какую информацию надо забыть, а какую оставить.



$$f_t = \sigma(W_f x_{t-1} + U_f h_{t-1} + b_f)$$

$$f_t \in [0,1]$$

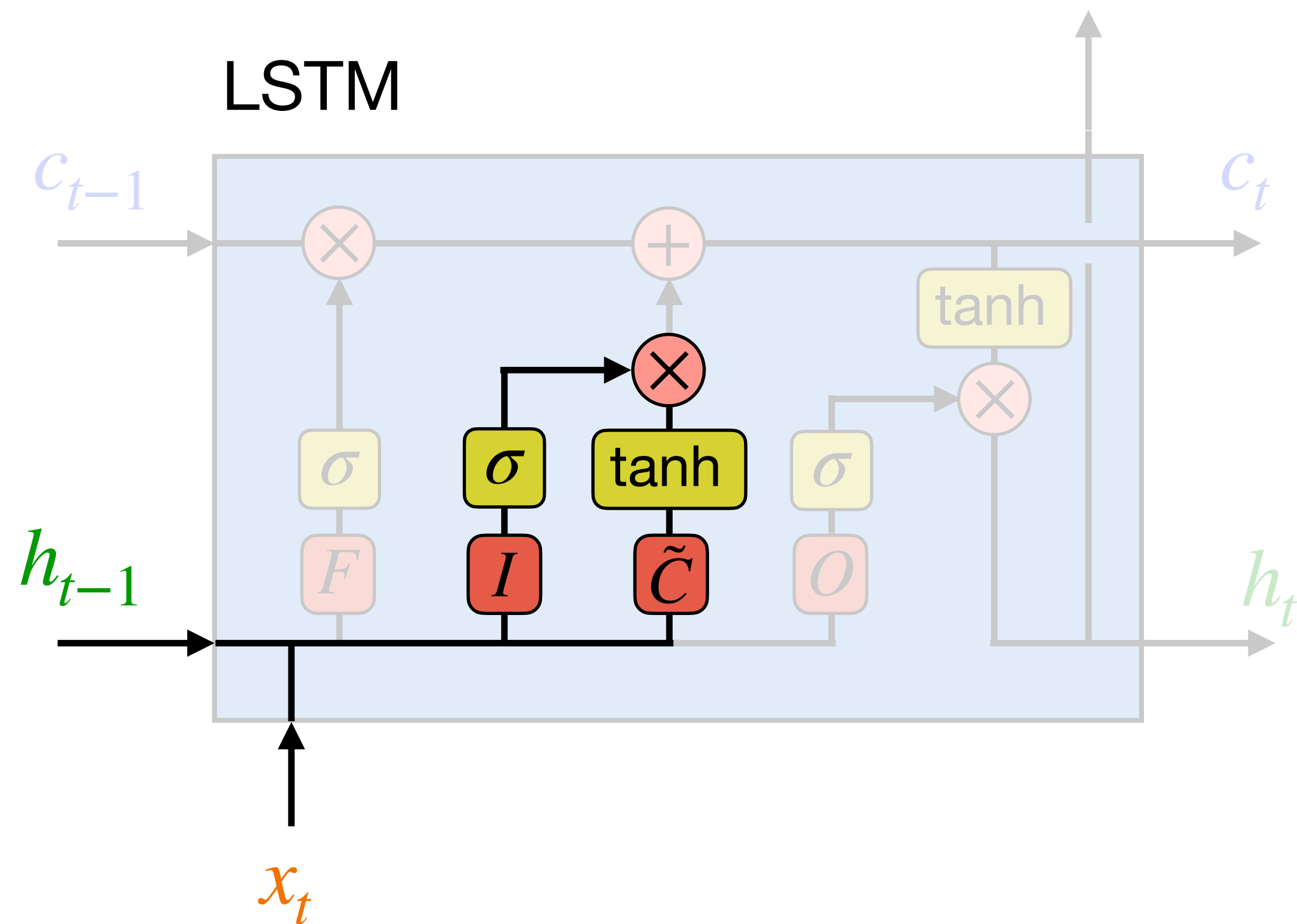
Еда была **вкусная** и мы **не разочаровались**.

x_3 – слово-маркер, можно забыть все до него.

x_7 – негативный маркер, но до него идет "**не**".
Знаем об этом из h_7 .

LSTM: фильтр входа

Контролирует, какую информацию надо добавить в вектор памяти c_t

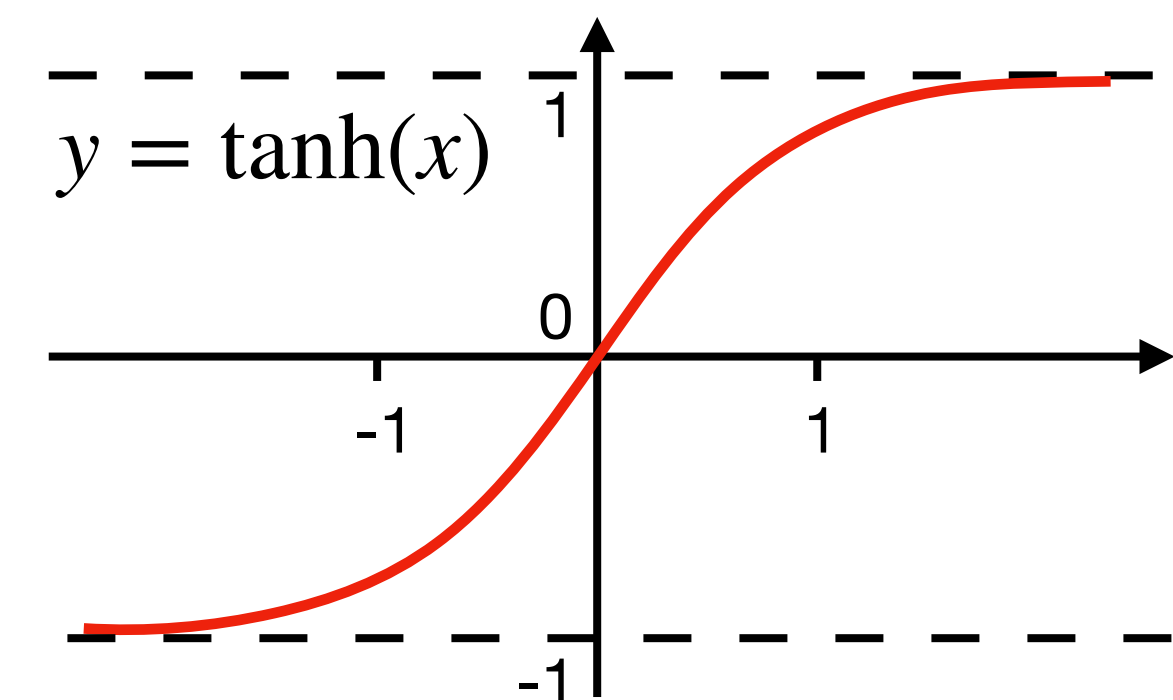


Из-за i_t можем не добавлять ничего

$$i_t = \sigma(W_i x_{t-1} + U_i h_{t-1} + b_i)$$

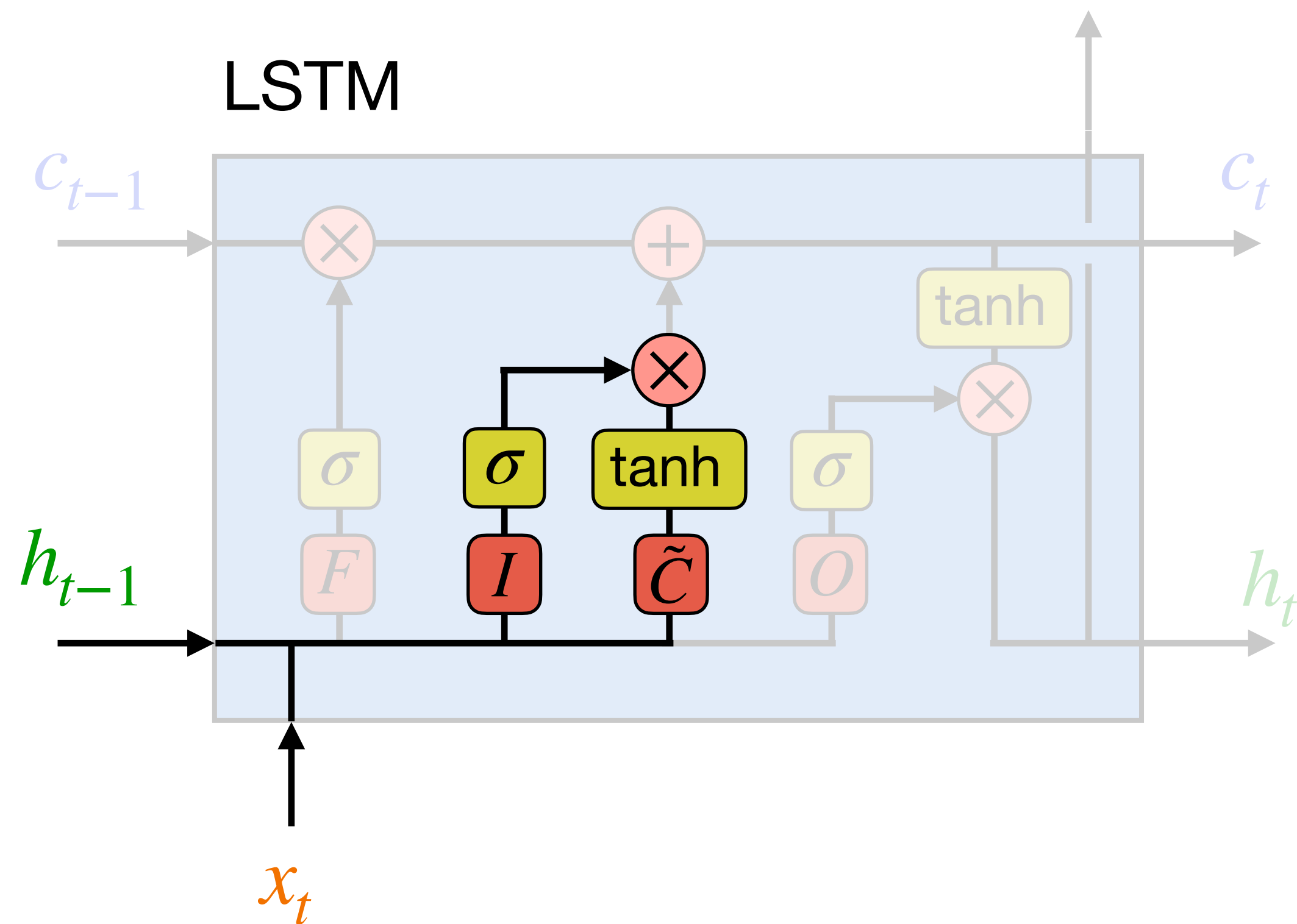
$$i_t \in [0, 1]$$

$$\tilde{c}_t = \tanh(W_c x_{t-1} + U_c h_{t-1} + b_c)$$



LSTM: фильтр входа

Контролирует, какую информацию надо добавить в вектор памяти c_t



Из-за i_t можем не добавлять ничего.

$$i_t = \sigma(W_i x_{t-1} + U_i h_{t-1} + b_i)$$

$$i_t \in [0, 1]$$

$$\tilde{c}_t = \tanh(W_c x_{t-1} + U_c h_{t-1} + b_c)$$

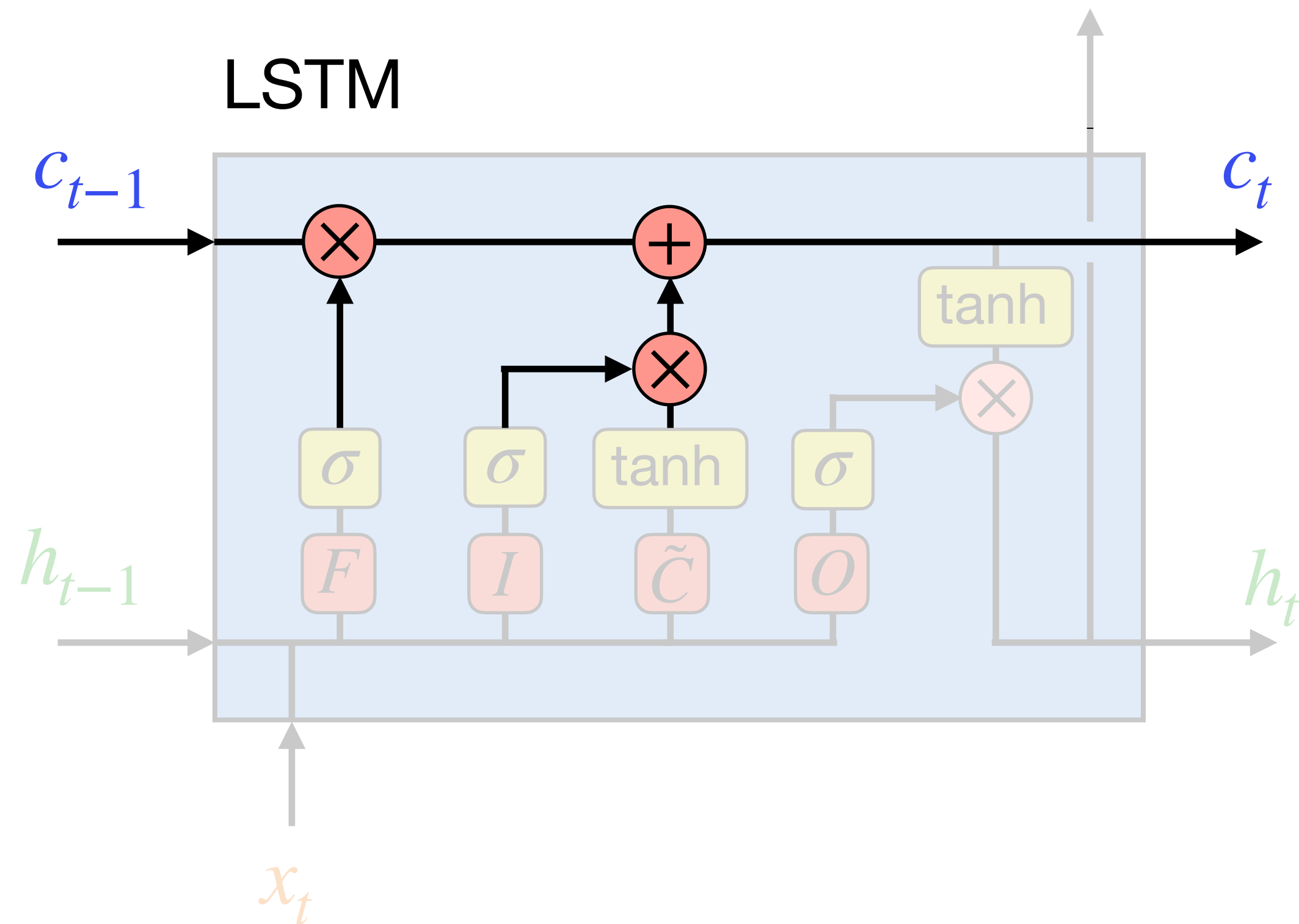
Еда была **вкусная** и **мы** не разочаровались.

x_3 – слово-маркер.
Запоминаем, что класс
положительный.

x_5 – не влияет на класс.
Ничего не добавляем.

LSTM: обновление памяти

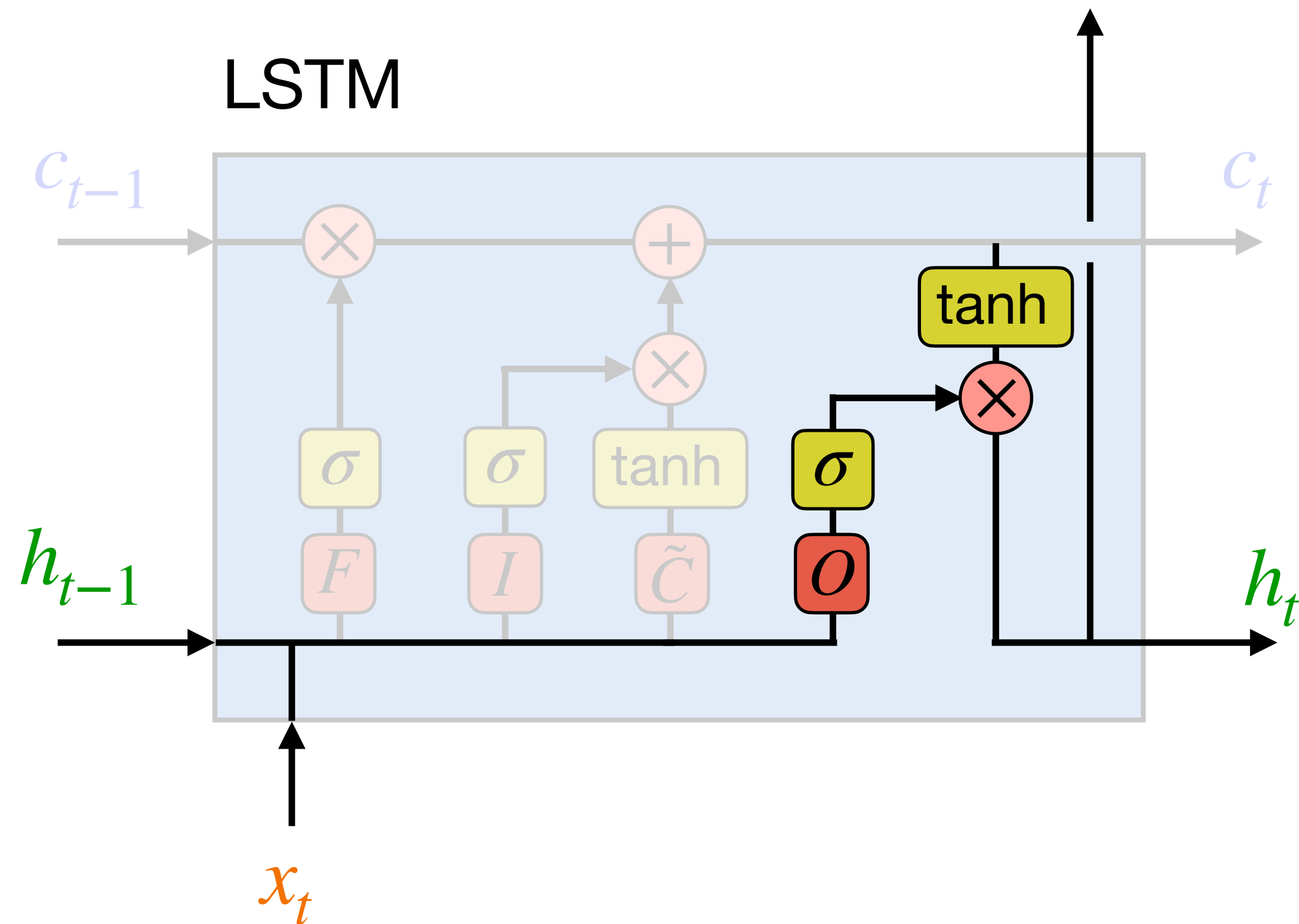
Удаляем ненужную информацию и добавляем новую.



$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

LSTM: фильтр выхода

Контролирует выход текущего шага.

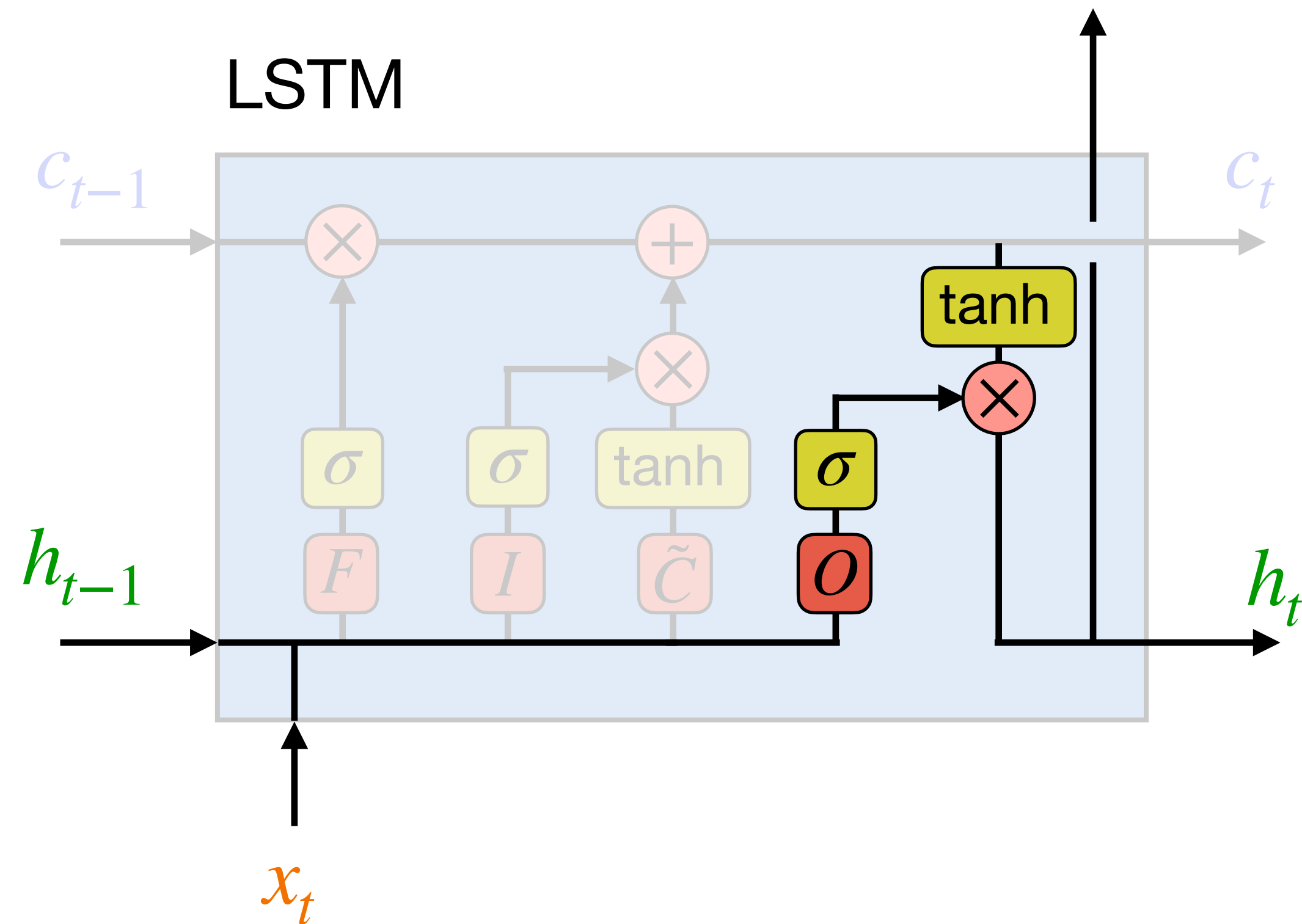


$$o_t = \sigma(W_o x_{t-1} + U_o h_{t-1} + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

LSTM: фильтр выхода

Контролирует выход текущего шага.



$$o_t = \sigma(W_o x_{t-1} + U_o h_{t-1} + b_o)$$

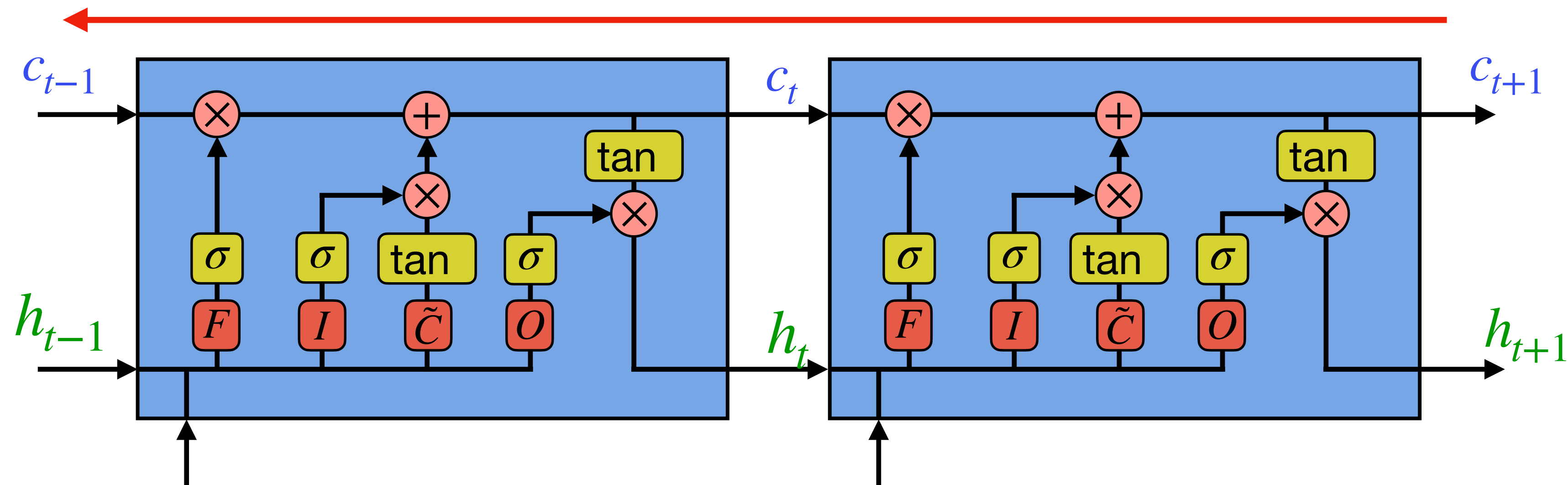
$$h_t = o_t \odot \tanh(c_t)$$

Учитель ведет урок, посвященный рекуррентным моделям. _

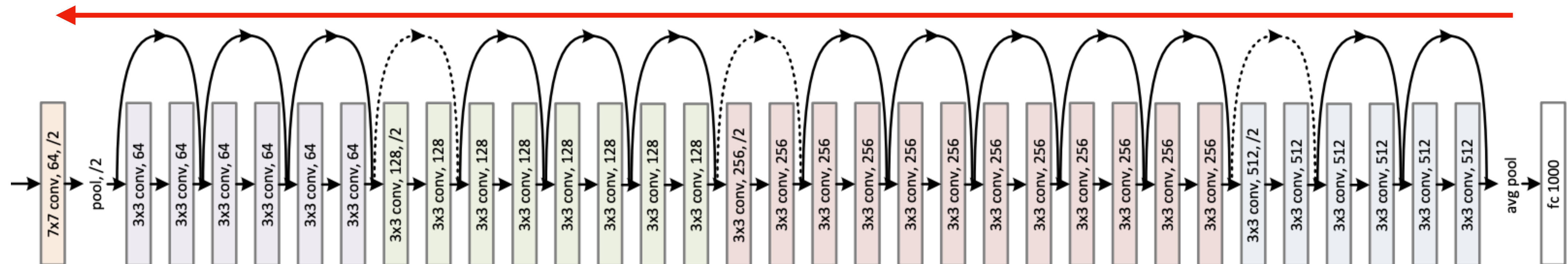
Начало нового предложения.
Надо вспомнить, что речь идет об учителе и уроке.

Градиенты не затухают

Градиенты свободно протекают через весь блок

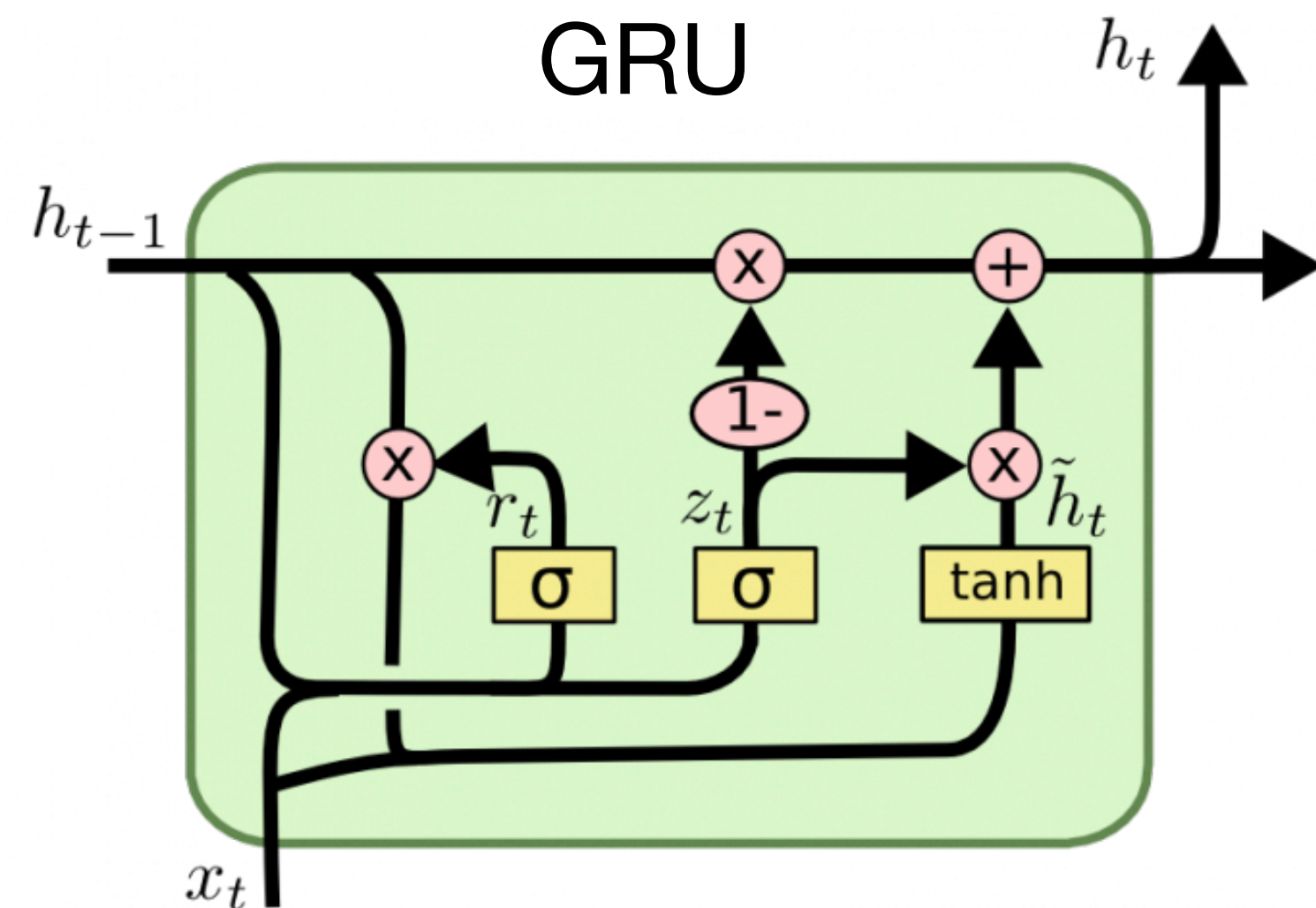


То же самое происходит в ResNet



Gated recurrent units (GRU)

Наиболее успешная уменьшенная вариация LSTM



$$z_t = \sigma(W_z x_{t-1} + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_{t-1} + U_r h_{t-1} + b_r)$$

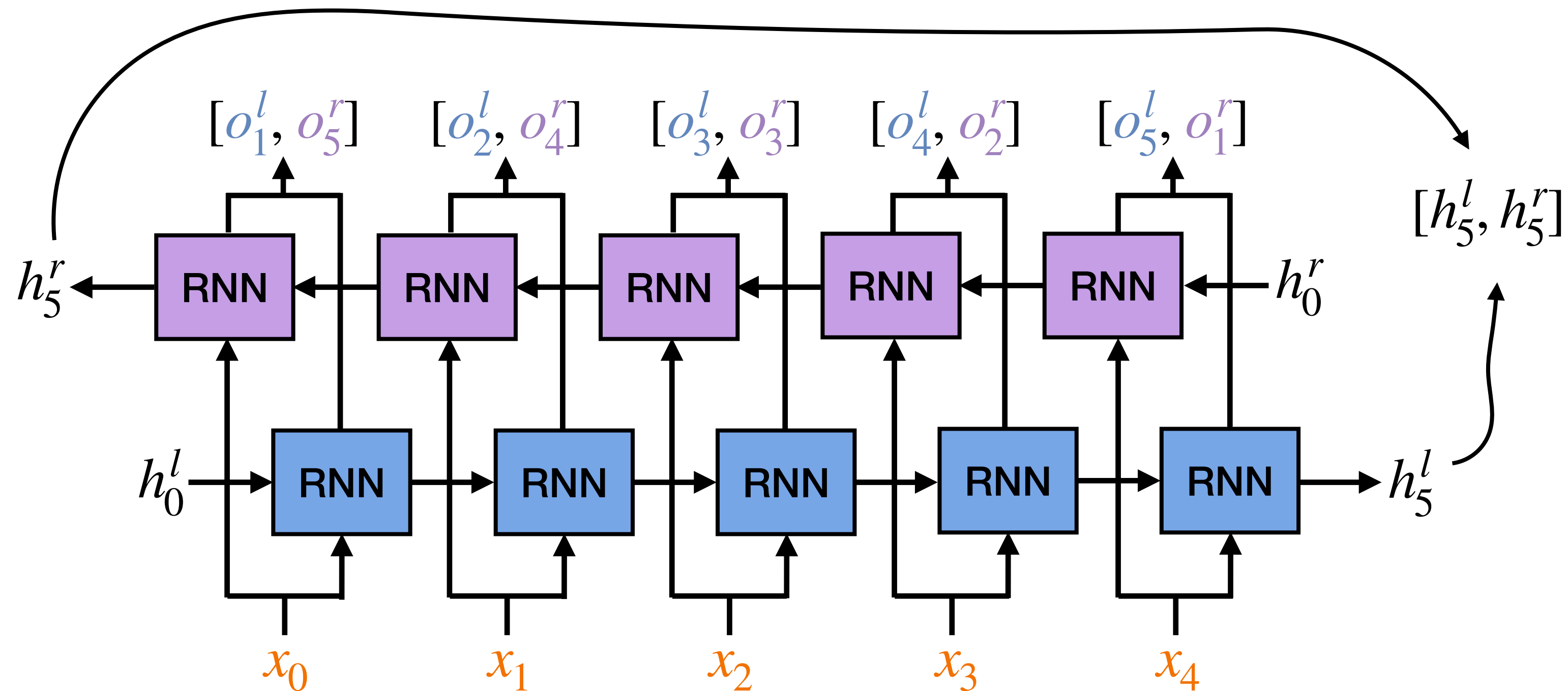
$$\tilde{h}_t = \tanh(W_h x_{t-1} + U_h (r_t \odot h_{t-1}) + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Фильтры забывания и входа объединены для уменьшения числа параметров

Двунаправленные рекуррентные сети (biRNN)

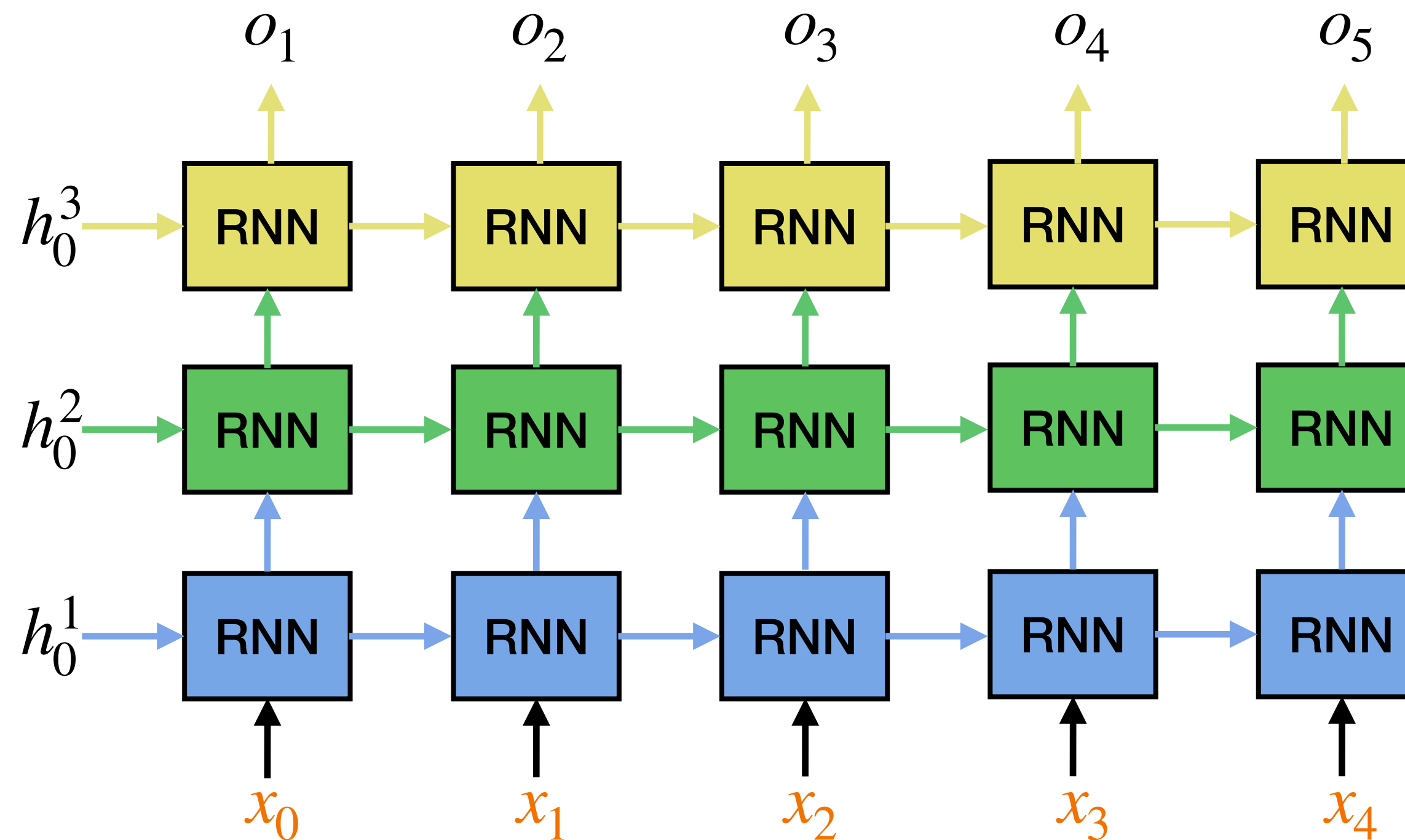
- Читает текст **слева направо** и **справа налево**.



- Имеет в два раза больше параметров.
- Бесполезна для задачи генерации.

Многослойные рекуррентные сети

- Выходы текущего слоя являются входами следующего.
- Извлекаются более сложные признаки.



- Число параметров увеличивается в число слоев раз.
- Обычно ограничиваются **двумя** слоями.